

UNIVERSITATEA DIN BUCUREȘTI



**FACULTATEA
DE
MATEMATICĂ
ȘI
INFORMATICĂ**



SPECIALIZAREA INFORMATICĂ

Lucrare de licență

Mindful Wallet - O soluție digitală pentru un consum responsabil

Absolvent

Ion Apelia-Cosmina

Coordonator științific

Prof. Mihai-Dumitru Prunescu

București, iunie 2024

Rezumat

Secolul 21 a venit cu multe inovații tehnologice. Odată cu dezvoltarea internetului, așa cum îl știm astăzi, a telefoanelor inteligente, a aplicațiilor și site-urilor de social media, omenirea a pășit într-o nouă realitate incitantă care însă vine la pachet cu o serie de neajunsuri, întrucât oamenii nu au reușit să se adapteze complet la acest nou stil de viață.

Internetul și social media au fost create pentru a conecta oamenii și pentru a-i ajuta să descopere lucruri noi, însă, din păcate, marile companii abuzează de aceste unelte pentru a-și crește cifra de vânzări. Social media a devenit un element cheie pentru promovarea produselor, iar achiziționarea produselor prezentate în mediul online nu a fost nicicând mai ușoară, totul fiind la un click distantă. Astfel, cumpărăturile online pot deveni cu ușurință un lucru nesănătos. Impulsul oamenilor de a cumpăra este tot mai mare, fiind alimentat de fluxul constant de reclame și de social media. Shoppingul online a devenit un mod de gratificare instantanee pentru majoritatea oamenilor, un mod ușor de a obține doza zilnică de serotonină, motiv pentru care am ajuns să trăim într-o societate în care consumul excesiv de produse a devenit o normalitate.

Aplicația web Mindful Wallet își propune să ofere utilizatorilor o platformă integrată pentru a gestiona și optimiza cheltuielile personale. Conceput pentru a promova un stil de viață sustenabil și pentru a contracara tendințele de consum excesiv, Mindful Wallet promite utilizatorilor săi funcționalități precum un jurnal de cheltuieli, rapoarte dedicate conturilor lor, pagini de sfaturi și trucuri pentru o mai bună gestionare a banilor, provocări, etc, care îi vor ajuta să își atingă mai ușor obiectivele financiare.

Utilizând tehnologiile Angular, .NET, Bootstrap și SQL, aplicația este o soluție completă ce vine în ajutorul utilizatorilor.

Cu Mindful Wallet, utilizatorii au la dispoziție un asistent digital care îi ajută să rămână conștienți și controlați în privința finanțelor lor, promovând decizii de cumpărare deliberate și evitând impulsurile costisitoare. Scopul final este de a facilita o relație mai sănătoasă cu banii și de a crea un echilibru între nevoia de satisfacție imediată și binele financiar pe termen lung.

Abstract

Cuvinte cheie: wallet, saving, spending, management, financial, Angular, .NET, SQL, Bootstrap

This thesis presents Mindful Wallet, a web application designed to foster mindful spending and saving habits among users, particularly addressing the challenges posed by excessive consumerism and impulsive online shopping behaviors. Developed using Angular, .NET, Bootstrap and SQL frameworks, the application features an expense log, spending reports, encouragement pages with money management tips, and personalized financial advice. Additionally, Mindful Wallet includes a goals page where users can set and track financial targets, such as saving for significant purchases.

Cuprins

Introducere.....	5
1.1 Contextul Proiectului	5
1.2 Ideea si scopul proiectului	6
1.3 Structura lucrării.....	6
Preliminarii	7
2.1 Tehnologii folosite	7
2.1.1 Angular.....	7
2.1.2 ASP.NET Core	9
2.1.3 Microsoft SQL Server	11
2.2 Descrierea tehnologică	13
2.2.1 Arhitectura aplicației	13
2.2.2 Securitatea datelor	17
Detalii implementare	22
3.1 Arhitectura	22
3.1.1 Backend.....	22
3.1.2 Diagrama de clase si funcții	24
3.1.3 Frontend	27
3.1.4 Diagrama de flux.....	29
3.2 Baza de date.....	30
3.2.1 Abordarea code first	30
3.2.2 Diagrama Entity-Relationship.....	32
3.2.3 Descrierea entităților și a relațiilor	32
3.3 Modul de funcționare al aplicației.....	34
3.3.1 User case diagram	34
3.3.2 Diagrama de interacțiuni user - frontend - backend	35
3.4 Implementarea Paginilor	37
3.4.1 Pagina de login.....	37
3.4.2 Pagina de signup.	42

Cuprins

3.4.3 Pagina Finanțe.....	43
3.4.4 Pagina Goals.....	52
3.4.5 Pagina Calendar.	55
3.4.6 Pagina Home.	57
3.4.7 Alte pagini	61
3.5 Testarea și validarea	62
Concluzii.....	64
4.1 Rezultate obtinute.....	64
4.2 Limitari si provocari	65
4.3 Sugestii pentru lucrari viitoare	65
Bibliografie.....	67

Introducere

1.1 Contextul Proiectului

În era digitală a secolului 21, avansurile rapide în tehnologie au transformat fundamental modul în care trăim, lucrăm și interacționăm. Odată cu dezvoltarea internetului, a dispozitivelor mobile, și a platformelor de social media, a avut loc o schimbare radicală în comportamentul de consum, deoarece acestea facilitează accesul la cumpărături online și promovează o cultură a gratificării imediate. Această transformare aduce beneficii în termeni de accesibilitate, rapiditate și eficiență, dar și noi provocări, precum consumul excesiv și gestionarea ineficientă a finanțelor personale.

Problema consumului excesiv este amplificată de strategiile de marketing și de omniprezența tehnologiei, care stimulează achizițiile impulsive și creează probleme financiare pentru o mare parte dintre oameni. Articolul „Excessive shopping on the internet: recent trends in compulsive buying-shopping disorder”¹ arată faptul că pentru o parte semnificativă din populație, cumpărăturile online au devenit un comportament nesănătos, o dependență compulsivă (CBSD - compulsive buying-shopping disorder) asociată cu utilizarea improprie internetului. Această tulburare se alătură altor probleme deja bine cunoscute legate de utilizarea nepotrivită a internetului, precum dependența de jocuri și utilizarea excesivă a rețelelor sociale.

Pentru mai multe detalii legate de CSBD, vezi [14] din bibliografie.

Aceste comportamente pot fi interpretate ca forme de evadare sau de căutare a satisfacției imediate, fenomene care sunt în creștere și care afectează starea de bine a indivizilor pe termen lung.

¹ Astrid Müller, Maithilee Joshi, Tobias A. Thomas, Excessive shopping on the internet: recent trends in compulsive buying-shopping disorder.

Ideea si scopul proiectului

Mindful Wallet este o aplicație web dezvoltată pentru a răspunde provocării cumpărăturilor în exces, oferind utilizatorilor o platformă integrată pentru gestionarea cheltuielilor personale. Aplicația promovează un stil de viață financiar sănătos și sustenabil, încurajând utilizatorii să reflecteze asupra achizițiilor lor și să facă alegeri informate.

Mindful Wallet oferă o platformă complexă, dar ușor de utilizat, care permite utilizatorilor:

- să înregistreze și urmărească cheltuielile: Utilizatorii pot introduce tranzacții pentru a avea o imagine clară și actualizată a finanțelor lor.
- să analizeze comportamentul de cumpărare: Aplicația oferă rapoarte personalizate care ajută la identificarea tendințelor de consum și la ajustarea comportamentelor de cheltuire.
- să stabilească și monitorizeze obiectivele financiare: Fie că este vorba de economisire pentru o vacanță, achiziția unei case sau crearea unui fond de urgență, Mindful Wallet încurajează stabilirea și urmărirea obiectivelor pe termen lung.
- să primească sfaturi personalizate: Pe baza datelor introduse și a obiceiurilor de consum, aplicația oferă recomandări și sfaturi pentru îmbunătățirea sănătății financiare.

1.2 Structura lucrării

Capitolele acestei lucrări sunt structurate după cum urmează: Preliminariile ce prezintă câteva aspecte teoretice cu privire la tehnologiile (framework-urile) folosite la construirea aplicației web dar și legate de arhitectura aplicației și modul de securizare a datelor utilizatorului.

Capitolul următor prezintă detaliile de implementare, adică modul de implementare propriu-zis al aplicației, începând cu arhitectura, baza de date și continuând cu implementarea acesteia, descrierea modurilor de utilizare, a interacțiunilor user-frontend-backend, și descrierea implementării fiecărei pagini, în frontend și respectiv backend. Capitolul se încheie cu testarea funcționalităților aplicației.

Capitolul final prezintă rezultatele obținute, limitările și provocările întâmpinate în dezvoltarea aplicației și se încheie cu idei de continuare a dezvoltării aplicației.

Preliminarii

2.1 Tehnologii folosite

Pentru a dezvolta o aplicație web complexă și robustă, integrarea armonioasă a tehnologiilor este esențială. Angular este utilizat pentru front-end, oferind un framework robust pentru crearea interfețelor de utilizator dinamice și interactive. Acesta permite dezvoltatorilor să construiască componente modulare și să gestioneze eficient interacțiunile cu utilizatorii. .NET servește ca back-end, gestionând logica de business și securitatea aplicației. Acesta facilitează crearea API-urilor RESTful care comunică între front-end și baza de date. Baza de date relațională SQL Server, stochează și gestionează datele aplicației, asigurând acces rapid și securizat la informații. Prin intermediul ORM-urilor precum Entity Framework, .NET interacționează fluid cu baza de date, simplificând operațiunile de citire, scriere și actualizare a datelor.

Angular, .NET și bazele de date relaționale sunt tehnologii care se completează reciproc și creează un ecosistem robust pentru dezvoltarea aplicațiilor web complexe.

Cateva dintre beneficiile asocierii acestor tehnologii sunt:

Separarea clară a responsabilităților ce implică o dezvoltare modulară a aplicației și la o întreținere ce se realizează cu ușurință. Angular se ocupă de partea de front-end, de interfața cu utilizatorul, în timp ce .NET gestionează logica și comunicarea cu serverul, iar baza de date a aplicației stochează și gestionează datele.

Nivelul de securitate sporit al aplicației, ce se realizează prin intermediul mecanismelor avansate ale .NET, și care se integrează ușor cu interfața creată în Angular.

Astfel, combinația acestor tehnologii permite dezvoltarea unei aplicații web performante, scalabile și ușor de întreținut, oferind utilizatorilor o experiență plăcută și sigură.

2.1.1 Angular

Angular este un cadru open-source pentru dezvoltarea aplicațiilor web, creat și întreținut de Google, destinat dezvoltării interfețelor cu utilizatorul (Front-end). Inițial lansat în 2010 sub numele de AngularJS, ce provenea de la limbajul utilizat, JavaScript. Angular a fost ulterior refăcut folosind TypeScript, un derivat al JavaScript, pentru a îmbunătăți structura și

funcționalitatea aplicațiilor web moderne.² Pentru mai multe informații vezi [1] din bibliografie.

AngularJS a fost lansat în 2010 și a introdus un nou mod de a construi aplicații web, prin extinderea limbajului HTML și aducerea unor funcționalități de server în partea de client. Această versiune inițială a fost rapid adoptată datorită capacităților sale de a crea aplicații dinamice și interactive. Cu toate acestea, pentru a face față cerințelor moderne, Google a decis să refacă complet framework-ul. Astfel, în 2016 a apărut Angular (cunoscut și ca Angular 2+), care este o versiune complet refăcută și îmbunătățită, bazată pe TypeScript .

Angular este construit în jurul unei arhitecturi bazate pe componente. Fiecare componentă este o unitate de cod autonomă, care înglobează fișierele HTML pentru vizualizare, CSS pentru stilizare și TypeScript pentru logică. Această modularitate permite dezvoltarea de aplicații complexe într-o manieră organizată și scalabilă .³ Vezi [4] din bibliografie.

Componenta de bază a unui proiect Angular este definită de regulă în fișierele `app.routes.ts` și `app.config.ts`. Aceste fișiere sunt esențiale pentru configurarea și gestionarea rutelor și setărilor aplicației. În mod specific:

- `app.routes.ts`: definește configurația rutelor pentru navigația în cadrul aplicației, permițând aplicației să suporte Single Page Applications (SPA).
- `app.config.ts`: conține diverse configurații ale aplicației, inclusiv setările de inițializare și configurările specifice mediului.

Caracteristici Cheie ale frameworkului Angular cuprind:

Extensibilitate și Flexibilitate: Angular permite extinderea și modificarea facilă a aplicațiilor datorită designului său modular. Fiecare componentă poate fi dezvoltată și testată independent, ceea ce simplifică mentenanța și adăugarea de noi funcționalități .

Testabilitate: Angular oferă suport robust pentru testarea unităților și testarea end-to-end, permițând dezvoltatorilor să detecteze și să corecteze defectele înainte ca utilizatorii să întâmpine probleme .

² Programare Frontend: Angular vs React. Disponibil la: <https://www.newtech.ro/blog-react-vs-angular/> [Accesat 02 Iunie 2024].

³ Freeman, A. (2020) 'Putting Angular in Context', in Pro Angular 9. 4th edn. London, UK: Apress, pp. 69–72.

Performanță: Angular utilizează atât compilarea Just-In-Time (JIT) cât și Ahead-Of-Time (AOT) pentru a optimiza performanța aplicațiilor. JIT compilează codul în timpul execuției, în timp ce AOT compilează codul în timpul procesului de build, rezultând în aplicații mai rapide și mai eficiente .

Suport pentru Single Page Applications (SPA): Angular este ideal pentru dezvoltarea aplicațiilor de tip SPA, unde întreaga aplicație este încărcată o singură dată, iar paginile sunt actualizate dinamic fără a reîncărca întregul site .

Angular CLI: Angular CLI (Command Line Interface) simplifică procesele de inițializare, dezvoltare, și testare a aplicațiilor, oferind comenzi pentru crearea componentelor, modulelor, și serviciilor, precum și pentru managementul proiectului și lansarea în producție .⁴ Pentru mai multe informații despre caracteristicile frameworkului Angular, vezi referința [3] din bibliografie.

Concluzionând, Angular rămâne unul dintre cele mai puternice și flexibile framework-uri pentru dezvoltarea aplicațiilor web moderne. Cu un ecosistem robust și o comunitate activă de dezvoltatori, Angular continuă să evolueze, oferind un set vast de unelte și capabilități pentru crearea de aplicații web complexe și performante. Fiind un framework susținut de Google și utilizat pe scară largă, Angular este o alegere solidă pentru orice proiect de dezvoltare front-end .

2.1.2 ASP.NET Core

ASP.NET este un framework ce ajută la construirea aplicațiilor web. Platforma cross si open source poate fi folosită pentru a construi rapid aplicații dinamice ce au renderizare pe server. Asp.NET poate fi folosit de asemenea pentru a crea API-uri (interfață de programare a aplicației) HTTP care pot fi utilizate de aplicații mobile, de aplicații web single-page (SPA) bazate pe browser, cum ar fi Angular și React, sau de alte aplicații de backend.⁵ Vezi [5] din bibliografie.

ASP.NET Core a fost dezvoltat pentru a înlocui ASP.NET, un framework bine stabilit în ecosistemul Windows. Lansat inițial în 2002 ca parte a .NET Framework 1.0, ASP.NET a evoluat prin multiple versiuni, fiecare aducând îmbunătățiri și noi funcționalități. Cu toate

⁴ Freeman, A. (2020) 'Getting Ready', in Pro Angular 9. 4th edn. London, UK: Apress, pp. 28–29.

⁵ Lock, A. (2021) ASP.NET Core In Action. 2nd edn. Shelter Island, NY: Manning, pp 46

acestea, ASP.NET era limitat de dependența sa de .NET Framework și de sistemul de operare Windows. ASP.NET Core a fost creat pentru a depăși aceste limitări, oferind o arhitectură modulară și suport cross-platform, permițând rularea pe Windows, macOS și Linux.⁶ Pentru mai multe detalii legate de evoluția acestui framework, vezi [10] din bibliografie.

Cateva dintre caracteristicile cheie ale platformei ASP.Net cuprind faptul că este o aplicație cross-platform, arhitectura modulară, performanța ridicată, suportul pentru containerele Docker etc.

ASP.NET Core funcționează pe Windows, Linux și macOS, permițând dezvoltarea aplicațiilor pe diverse platforme. Arhitectura sa modulară permite utilizarea doar a componentelor necesare, menținând aplicațiile compacte și eficiente. Cu serverul web Kestrel, ASP.NET Core oferă performanțe ridicate și este unul dintre cele mai rapide framework-uri datorită optimizărilor arhitecturii sale lightweight. Este optimizat pentru containere Docker, facilitând implementarea în medii cloud. Integrează funcționalități built-in pentru autentificare și autorizare, alături de un sistem de configurare robust. ASP.NET Core utilizează un pipeline de middleware pentru gestionarea cererilor HTTP, adăugând funcționalități precum gestionarea erorilor și autentificarea. Aceste caracteristici fac din ASP.NET Core un framework versatil și performant pentru dezvoltarea aplicațiilor moderne.

„Aplicațiile web tradiționale bazate pe pagini și cu renderizare pe server sunt baza dezvoltării ASP.NET, atât cu versiunea anterioară de ASP.NET, cât și cu ASP.NET Core. În plus, aplicațiile single-page (SPA), care folosesc un framework pe partea de client ce comunică de obicei cu un server REST, sunt ușor de creat cu ASP.NET Core. Indiferent dacă folosești Angular, Vue, React sau un alt framework pe partea de client, este ușor să creezi o aplicație ASP.NET Core care să acționeze ca API-ul de server.” (Lock, 2021, pp. 56,57) Vezi [6] din bibliografie pentru sursa.

Construirea unei aplicații ASP.NET Core începe de obicei cu crearea unui nou proiect folosind un șablon predefinit. Proiectul rezultat include fișiere și directoare de bază esențiale pentru funcționarea aplicației:

- Program.cs: configurează și pornește aplicația.
- Startup.cs: definește pipeline-ul de middleware și configurează serviciile aplicației.

⁶ Sharma, A. K., 2021. *Evolution / History of ASP.NET*. Disponibil la: <https://www.ittutorialswithexample.com/2021/03/evolution-history-of-aspnet.html> [Accesat 02 Iunie 2024].

- appsettings.json: conține setările de configurare.
- Controllers: folderul în care sunt definite controllerele MVC sau API.

ASP.NET Core suportă două modele principale pentru generarea de răspunsuri: Razor Pages și MVC. Razor Pages este recomandat pentru aplicații server-side rendered, în timp ce MVC este folosit pentru crearea API-urilor

Avantajele Utilizării ASP.NET Core

- **Flexibilitate și Scalabilitate:** Datorită arhitecturii modulare, aplicațiile ASP.NET Core sunt ușor de extins și scalat. Fiecare componentă poate fi adăugată sau eliminată fără a afecta restul aplicației
- **Ecosistemul Dezvoltatorilor:** ASP.NET Core beneficiază de un ecosistem puternic, susținut de Microsoft și de o comunitate activă de dezvoltatori. Documentația extensivă și resursele online facilitează învățarea și rezolvarea problemelor
- **Suport pe Termen Lung:** Microsoft oferă suport pe termen lung (LTS) pentru versiunile ASP.NET Core, asigurând actualizări de securitate și corecții de erori pentru cel puțin trei ani de la lansare.

Putem concluziona spunând că ASP.NET Core reprezintă un salt semnificativ față de framework-urile anterioare, oferind un set robust de unelte și funcționalități pentru dezvoltarea aplicațiilor web moderne. Cu suport cross-platform, performanță ridicată și o arhitectură modulară, ASP.NET Core este alegerea ideală pentru dezvoltatorii care doresc să construiască aplicații web eficiente și scalabile.

2.1.3 Microsoft SQL Server

Microsoft SQL Server este un sistem de management al bazelor de date relaționale (RDBMS) dezvoltat de Microsoft. SQL Server este utilizat pentru a stoca și a gestiona informații și permite accesul rapid și securizat la date. Acesta este unul dintre cele mai populare și robuste sisteme de baze de date disponibile, oferind o gamă largă de funcționalități pentru a susține diverse nevoi de afaceri și tehnologice.

Microsoft SQL Server a fost lansat pentru prima dată în 1989, ca rezultat al unui parteneriat între Microsoft și Sybase. Versiunile inițiale au fost dezvoltate pentru platforma OS/2, dar în 1993, Microsoft a preluat controlul complet al proiectului și a lansat SQL Server 4.2 pentru Windows NT. De atunci, SQL Server a trecut prin multiple iterații, fiecare aducând

îmbunătățiri semnificative în performanță, securitate și funcționalitate.⁷ Pentru mai multe informații legate de istoricul SQL server, vezi [2] din bibliografie.

Caracteristicile cheie cuprind:

- **Motorul de Bază de Date** este serviciul central al SQL Server, responsabil pentru stocarea, procesarea și securizarea datelor. Acesta oferă acces controlat la date și asigură integritatea acestora prin procesarea tranzacțiilor. Funcționalități precum indexarea, procesarea în memorie și capabilitățile avansate de interogare contribuie la eficiența și performanța generală a sistemului, facilitând manipularea rapidă și sigură a datelor.
- SQL Server integrează **capabilități de machine learning** prin suportul pentru limbaje precum R și Python. Aceste servicii permit utilizatorilor să creeze, să antreneze și să implementeze modele de învățare automată direct în cadrul SQL Server. Aceasta face posibilă analiza predictivă și extragerea de informații valoroase din datele stocate, fără a fi necesară migrarea acestora în alte platforme de analiză.
- **Serviciile de Integrare SQL Server (SSIS)** oferă o platformă robustă pentru construirea soluțiilor de integrare și transformare a datelor la nivel de întreprindere. SSIS facilitează procesele ETL (Extract, Transform, Load), esențiale pentru depozitarea datelor și pentru transferul datelor între diverse sisteme. Aceste servicii asigură că datele sunt curate, consistente și disponibile pentru analize complexe.
- **SQL Server Analysis Services (SSAS)** furnizează instrumente pentru procesarea datelor analitice, esențiale pentru aplicațiile de business intelligence. SSAS suportă modele de date OLAP (Online Analytical Processing) și tabulare, care permit utilizatorilor să efectueze analize complexe și să genereze rapoarte detaliate. Aceasta ajută companiile să ia decizii informate pe baza datelor analizate.
- **Serviciile de Raportare SQL Server (SSRS)** reprezintă o soluție completă pentru crearea, implementarea și gestionarea rapoartelor. SSRS permite integrarea diverselor surse de date și suportă o varietate de formate de rapoarte. Utilizatorii pot crea rapoarte interactive și programate, facilitând distribuirea informațiilor critice în cadrul organizației.

⁷ Awati, R. & Hughes, A., n.d. Microsoft SQL Server. Disponibil la: <https://www.techtarget.com/searchdatamanagement/definition/SQL-Server> [Accesat 02 Iunie 2024]

- **Replicarea** în SQL Server implică tehnologii pentru copierea și distribuirea datelor între diferite baze de date, asigurând consistența acestora în sisteme distribuite. Replicarea suportă atât tranzacții curente cât și snapshot-uri, permițând companiilor să mențină date actualizate și să asigure continuitatea afacerilor în caz de întreruperi.
- **Serviciile de Calitate a Datelor (DQS)** oferă instrumente pentru curățarea și deduplicarea datelor. Aceste servicii ajută la menținerea calității datelor prin procese bazate pe cunoștințe, asigurând că datele sunt corecte, complete și utilizabile pentru analize ulterioare. DQS contribuie la îmbunătățirea fiabilității și acurateței datelor.
- **Serviciile de Date Master (MDS)** gestionează și mențin datele master, care sunt datele critice pentru funcționarea unei afaceri. MDS asigură că aceste date sunt consistente și fiabile, facilitând gestionarea centralizată a informațiilor importante pentru afaceri. Aceasta contribuie la îmbunătățirea proceselor decizionale și a coordonării între departamente.⁸

Pentru mai multe informații legate de caracteristicile SQL Server, vezi [8] din bibliografie.

Microsoft SQL Server este un sistem de management al bazelor de date relaționale robust și versatil, care oferă o gamă largă de funcționalități pentru a răspunde nevoilor diverse ale organizațiilor, fiind un tool potrivit pentru integrarea în aproape orice fel de aplicație web. Cu o arhitectură modulară, performanță ridicată, securitate avansată și suport extins pentru cloud, SQL Server continuă să fie un lider de piață în domeniul managementului bazelor de date.

2.2 Descrierea tehnologică

2.2.1 Arhitectura aplicației

Arhitectura aplicației MindfulWallet joacă un rol crucial în asigurarea succesului pe termen lung al proiectului. O arhitectură bine definită nu doar facilitează dezvoltarea inițială, ci și permite scalabilitatea, mentenabilitatea și extensibilitatea aplicației pe măsură ce cerințele utilizatorilor și tehnologia evoluează. În special, prin adoptarea unei arhitecturi bazate pe componente în cadrul unui proiect Angular și a arhitecturii Clean Architecture în .NET, se

⁸ Ray, M. & West, R., 2024. What is SQL Server?. Disponibil la: <https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server?view=sql-server-ver16> [Accesat 02 iunie 2024].

realizează o separare clară a preocupărilor, promovând reutilizabilitatea codului și reducând complexitatea. Aceasta abordare metodică și structurată contribuie la crearea unei aplicații robuste, flexibile și ușor de testat, capabilă să răspundă prompt și eficient provocărilor viitoare.

2.2.1.1 Arhitectura Backend

Clean Architecture este un stil de arhitectură software propus de Robert C. Martin, cunoscut și sub numele de Uncle Bob. Acesta se concentrează pe separarea responsabilităților în cadrul aplicațiilor software, facilitând astfel mentenanța, testarea și evoluția acestora. În contextul dezvoltării cu .NET, Clean Architecture oferă o metodă structurată de organizare a codului, asigurând o separare clară între diferitele componente ale aplicației.

Principiile Clean Architecture

1. **Independența de Framework-uri:** Aplicația trebuie să fie ușor de utilizat și testat fără a depinde de un framework specific.
2. **Independența de UI:** Logica de business trebuie să fie separată de interfața utilizatorului, astfel încât UI-ul să poată fi schimbat fără a afecta restul aplicației.
3. **Independența de Baze de Date:** Aplicația trebuie să poată schimba baza de date fără a afecta logica de business.
4. **Independența de Agenți Externi:** Interacțiunile cu sistemele externe (API-uri, biblioteci terțe) trebuie izolate, astfel încât acestea să poată fi schimbate ușor.

Clean Architecture este organizată în mai multe straturi, fiecare având roluri și responsabilități bine definite:

- **Entități:** Acestea reprezintă obiectele de business ale aplicației, conțin logica centrală și regulile de business și sunt independente de framework-uri sau infrastructură.
- **Cazuri de Utilizare (Use Cases):** Definește operațiile aplicației și conțin logica specifică pentru fiecare operație, fiind orchestrate de serviciile de aplicație.
- **Interfață (Interface Adapters):** Acest strat conține adaptori care fac legătura între cazurile de utilizare și alte straturi (ex. UI, baze de date), transformând datele dintr-un format în altul.
- **Infrastructură (Infrastructure):** Conține codul specific framework-urilor și detaliile de implementare pentru accesul la baza de date, interacțiunile cu sistemele externe și alte

aspecte tehnice.⁹ Pentru a accesa sursa și o descriere mai amănunțită a arhitecturii de tip clean, accesează [7] din bibliografie.

Implementarea Clean Architecture în .NET presupune organizarea proiectului în mai multe module distincte, fiecare corespunzând unui strat specific:

1. **Core:** Include entitățile, interfețele și cazurile de utilizare. Acesta reprezintă nucleul aplicației și este complet independent de framework-uri și infrastructură.
2. **Application:** Conține serviciile care implementează cazurile de utilizare și obiectele de transfer de date între straturi.
3. **Infrastructure:** Cuprinde implementările concrete ale repository-urilor, contextul bazei de date și serviciile de acces la sisteme externe.
4. **Presentation:** Include componentele de interfață, cum ar fi API-urile și interfețele grafice, fiind responsabile pentru interacțiunea cu utilizatorii finali.

Clean Architecture oferă un cadru robust pentru dezvoltarea aplicațiilor în .NET, asigurând separarea clară a responsabilităților și facilitând mentenanța și evoluția aplicației. Prin organizarea codului în straturi distincte și respectarea principiilor de independență, dezvoltatorii pot crea aplicații scalabile, ușor de întreținut și adaptabile la schimbările tehnologice și de business. Această abordare nu doar îmbunătățește calitatea codului, dar și eficiența echipei de dezvoltare, conducând la produse software mai robuste și mai ușor de gestionat pe termen lung.

2.2.2.1.2 Arhitectura Frontend

Component-Based Architecture (CBA) reprezintă un stil de arhitectură software în care aplicațiile sunt dezvoltate prin combinarea unor componente reutilizabile și independente. Această abordare facilitează gestionarea complexității, îmbunătățește modularitatea și permite o dezvoltare mai rapidă și mai eficientă a aplicațiilor. În esență, CBA promovează dezvoltarea software-ului într-un mod similar construcției cu blocuri, unde fiecare componentă reprezintă un bloc care poate fi combinat și reutilizat în diverse aplicații.

⁹ Martin, R. C., 2012. The Clean Architecture. Disponibil la: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> [Accesat 02 Iunie 2024].

Principiile Component-Based Architecture

- **Reutilizarea Componentelor:** Unul dintre principalele avantaje ale CBA este reutilizarea componentelor. Odată create, componentele pot fi utilizate în multiple aplicații sau module, economisind timp și resurse în procesul de dezvoltare.
- **Independența Componentelor:** Componentele sunt proiectate să fie independente și autonome. Aceasta înseamnă că fiecare componentă își gestionează propria stare și logică și poate funcționa singură sau în combinație cu alte componente.
- **Interfațe Clare și Precise:** Fiecare componentă expune o interfață clară și bine definită, care specifică modul în care alte componente sau părți ale sistemului pot interacționa cu ea. Aceste interfețe facilitează integrarea și comunicarea între componente.
- **Încapsularea:** Componentele își ascund detaliile de implementare și expun doar funcționalitatea necesară prin interfețe publice. Aceasta protejează integritatea componentelor și permite modificarea lor fără a afecta restul sistemului.
- **Scalabilitatea și Extensibilitatea:** CBA permite dezvoltarea sistemelor scalabile și extensibile, unde noi funcționalități pot fi adăugate prin introducerea unor noi componente fără a modifica componentele existente.

Avantajele Component-Based Architecture

- **Modularitate:** Componentizarea sistemului facilitează înțelegerea, dezvoltarea și mentenanța acestuia. Fiecare componentă poate fi dezvoltată și testată independent, reducând astfel complexitatea generală a aplicației.
- **Flexibilitate:** Componentele independente pot fi ușor înlocuite sau actualizate fără a afecta alte părți ale sistemului. Aceasta permite adaptarea rapidă la cerințe noi sau schimbări tehnologice.
- **Timp de Dezvoltare Redus:** Reutilizarea componentelor existente accelerează procesul de dezvoltare, deoarece nu este necesar să se creeze fiecare componentă de la zero. Echipele pot reutiliza componente testate și verificate, reducând astfel timpul și efortul necesar pentru dezvoltare.
- **Îmbunătățirea Calității:** Prin reutilizarea componentelor deja testate și stabilite, se reduce riscul de erori și bug-uri. Acest lucru contribuie la creșterea fiabilității și stabilității sistemului.

- **Facilizarea colaborării în echipă:** În proiectele mari, diferite echipe pot lucra simultan la componente diferite, facilitând colaborarea și integrarea. Fiecare echipă poate fi responsabilă pentru dezvoltarea și mentenanța anumitor componente, îmbunătățind astfel eficiența și productivitatea.¹⁰

Pentru mai multe informații legate de component-based architecture, vezi referința [12] din bibliografie

2.2.2 Securitatea datelor

În era digitală contemporană, securitatea datelor utilizatorului a devenit o prioritate esențială pentru companii, guverne și indivizi deopotrivă. Creșterea exponențială a volumului de date generate și stocate online a adus cu sine provocări semnificative în ceea ce privește protecția informațiilor sensibile și confidențiale. Atacurile cibernetice, scurgerile de date și utilizarea necorespunzătoare a informațiilor personale sunt doar câteva dintre amenințările care subliniază importanța implementării unor măsuri robuste de securitate a datelor.

Securitatea datelor utilizatorului implică un set complex de practici, tehnologii și reglementări menite să protejeze informațiile împotriva accesului neautorizat, modificării sau distrugerii.

2.2.2.1 Autentificare JWT

JSON Web Tokens (JWT) a devenit una dintre cele mai populare metode de autentificare datorită simplității și eficienței sale. JWT este un standard deschis pentru crearea de tokenuri de acces care permit schimbul de informații între părți într-un mod securizat. JWT-urile sunt utilizate pe scară largă pentru a autentifica utilizatorii și a proteja resursele sensibile ale aplicațiilor.

Un JSON Web Token (JWT) este un obiect compact și autonom care transmite informații între părți ca un obiect JSON. Tokenul este compus din trei părți principale:

¹⁰Component-Based Architecture, Disponibil la: https://www.tutorialspoint.com/software_architecture_design/component_based_architecture.htm [Accesat 01 Iunie 2024].

1. **Header:** Conține metadata despre tipul tokenului (de obicei JWT) și algoritmul folosit pentru semnătură (de exemplu, HMAC SHA256).
2. **Payload:** Conține revendicările (claims), care sunt declarații despre o entitate (de obicei utilizatorul) și date adiționale. Există trei tipuri de revendicări:
Registered claims care sunt predefinite în specificația JWT și includ informații precum issuer, subject, audience și expiration. **Public claims** pot fi definite de utilizatori pentru a partaja informații între părți, dar trebuie să fie evitate coliziunile prin utilizarea de namespace-uri. **Private claim-urile** sunt utilizate în scopuri specifice aplicației și definite de utilizatori.
3. **Signature:** Verifică autenticitatea mesajului. Aceasta se obține prin luarea header-ului și payload-ului, concatenarea lor, aplicarea algoritmului specificat și apoi criptarea rezultatului cu o cheie secretă sau privată.

Cum funcționează JWT?

Când un utilizator se loghează, serverul generează un JWT care conține informații despre utilizator. Acest token este semnat cu o cheie secretă cunoscută doar de server. Tokenul generat este trimis clientului, de obicei în header-ul răspunsului HTTP sau ca un cookie.

Clientul stochează tokenul (de exemplu, în localStorage sau în cookies) și îl trimite în header-ul cererilor HTTP ulterioare către server.

Serverul verifică semnătura tokenului la fiecare cerere pentru a se asigura că este autentic și nu a fost modificat. Dacă semnătura este validă, serverul procesează cererea și furnizează accesul la resursele solicitate.

Avantajele utilizării JWT

1. JWT-urile sunt **compacte** și pot fi transmise prin URL-uri, parametri POST sau în header-urile HTTP. Fiind **autonome**, ele conțin toate informațiile necesare despre utilizator și nu necesită consultarea unei baze de date pentru fiecare cerere.
2. JWT-urile sunt semnate folosind **algoritmi criptografici**, ceea ce le face dificil de falsificat. Ele pot fi, de asemenea, criptate pentru a proteja informațiile sensibile din payload.
3. JWT-urile sunt ideale pentru aplicațiile distribuite, deoarece nu necesită stocarea sesiunilor pe server. Aceasta reduce încărcarea serverului și facilitează scalarea orizontală.

Dezavantaje și Provocări

1. Odată ce un JWT a fost emis, acesta nu poate fi revocat înainte de expirarea sa, ceea ce poate reprezenta un risc de securitate dacă un token este compromis.
2. Tokenurile cu durată de viață lungă pot reprezenta un risc de securitate, iar tokenurile cu durată de viață scurtă pot necesita mecanisme suplimentare de reînnoire, complicând implementarea.¹¹

Pentru mai multe detalii legate de autentificarea cu ajutorul JWT, vezi [13] din bibliografie.

Autentificarea cu JWT oferă o soluție simplă, sigură și eficientă pentru gestionarea autentificării și autorizării utilizatorilor în aplicațiile moderne. Adoptarea JWT poate îmbunătăți performanța și scalabilitatea aplicațiilor, dar necesită și o înțelegere atentă a riscurilor și implementarea unor măsuri de securitate adecvate pentru a proteja împotriva potențialelor vulnerabilități.

2.2.2.2 Hashingul parolelor

Protejarea parolelor utilizatorilor este esențială pentru securitatea cibernetică. Password hashing reprezintă o metodă fundamentală pentru securizarea parolelor stocate și prevenirea accesului neautorizat la datele sensibile. Password hashing este un proces prin care o parolă text simplu (plaintext) este transformată într-o secvență criptografică (hash), utilizând un algoritm specific. Această secvență criptografică nu poate fi transformată înapoi în textul original, ceea ce asigură confidențialitatea parolei.

Ce este Password Hashing?

Password hashing este un proces prin care o parolă de utilizator este trecută printr-un algoritm de hashing, rezultând un hash criptografic unic. Hash-ul generat este o reprezentare matematică a parolei originale și are câteva proprietăți esențiale:

1. **Unicitate:** Fiecare parolă produce un hash unic.
2. **Determinism:** Aceeași parolă va produce întotdeauna același hash atunci când este procesată prin același algoritm.

¹¹Introduction to JSON Web Tokens. Disponibil la: <https://jwt.io/introduction> [Accesat 3 Iunie 2024].

3. **Imposibilitatea de inversare:** Hash-ul nu poate fi transformat înapoi în parola originală.

Importanța Password Hashing-ului

Hash-urile protejează parolele utilizatorilor chiar și în cazul în care baza de date este compromisă. Fără hashing, parolele stocate în text simplu ar fi vulnerabile la atacuri.

Algoritmii de hashing sunt concepuți pentru a fi computațional intensivi, ceea ce face dificil pentru atacatori să ghicească sau să spargă parolele utilizând metode automate.

Algoritmi de Hashing Populari

- **MD5:** Un algoritm vechi, considerat acum nesigur din cauza vulnerabilităților cunoscute.
- **SHA-1:** Un alt algoritm mai vechi, care, de asemenea, nu mai este recomandat din cauza vulnerabilităților.
- **SHA-256:** Mai sigur decât predecesorii săi, dar nu ideal pentru hashing-ul parolelor din cauza vitezei sale rapide.
- **BCrypt:** Un algoritm robust, conceput special pentru hashing-ul parolelor, care include un factor de cost configurabil pentru a îmbunătăți securitatea.
- **Argon2:** Algoritmul de hashing câștigător al Password Hashing Competition din 2015, recunoscut pentru securitatea și flexibilitatea sa.

Salting și Peppering

Un salt este o valoare aleatorie adăugată la o parolă înainte de hashing. Saltul previne utilizarea atacurilor de tip rainbow table și asigură că hash-urile pentru parole identice sunt diferite.

Similar cu salting, peppering implică adăugarea unei valori secrete (pepper) la parola înainte de hashing. Pepper-ul este cunoscut doar de server și adaugă un strat suplimentar de securitate.

Implementarea și Managementul Hashing-ului de parole

Este crucial să se utilizeze un algoritm de hashing, cum ar fi BCrypt sau Argon2, care sunt lente și includ un factor de cost pentru a mări dificultatea atacurilor brute force.

Pentru fiecare parolă, un salt unic și aleatoriu trebuie generat și stocat împreună cu hash-ul. Acesta este esențial pentru a preveni atacurile cu rainbow table.

Securitatea algoritmilor de hashing se poate diminua în timp. Este important să se monitorizeze evoluția securității și să se migreze la algoritmi mai siguri dacă este necesar.¹²

Pentru mai multe detalii legate de password hashing, vezi [11] din bibliografie.

Password hashing este o practică esențială pentru protejarea parolelor utilizatorilor în orice sistem digital. Prin utilizarea unor algoritmi de hashing puternici și tehnici suplimentare precum salting și peppering, organizațiile pot asigura că datele utilizatorilor sunt protejate împotriva accesului neautorizat și atacurilor cibernetice. Implementarea corectă a hashing-ului contribuie semnificativ la securitatea generală a sistemelor informatice și la protejarea informațiilor sensibile ale utilizatorilor.

¹² Team, S., 2022. What is password hashing?. Disponibil la: <https://stytch.com/blog/what-is-password-hashing/> [Accesat 03 Iunie 2024].

Detalii implementare

3.1 Arhitectura

3.1.1 Backend

În cadrul proiectului MindfulWalletAPI, am aplicat arhitectura de tip Clean Architecture pentru a asigura o separare clară a responsabilităților între diferitele straturi ale aplicației. Clean Architecture, cunoscută și sub denumirile de Hexagonal Architecture sau Onion Architecture, are ca scop principal decuplarea diferitelor părți ale aplicației, astfel încât modificările într-un strat să nu afecteze alte straturi. Aceasta abordare promovează mentenabilitatea, testabilitatea și scalabilitatea aplicației.

Structura Proiectului:

MindfulWallet.API: Acesta corespunde stratului de prezentare, responsabil pentru expunerea API-ului web și manipularea cererilor HTTP. Cuprinde controllere (AccountController.cs, EventController.cs, ExpenseController.cs, FinanceController.cs, GoalController.cs, ReportController.cs, UserController.cs) și documente de configurare și lansare a aplicației: *appsettings.json* face configurațiile aplicației, iar *Program.cs* este punctul de intrare al aplicației, care la randul său face configurarea și lansarea aplicației.

MindfulWallet.Application: Acest strat conține logica de aplicație, cazurile de utilizare și serviciile care sunt decuplate de infrastructura specifică.

În aplicația mea, acest strat cuprinde un folder de servicii de aplicație ce reprezintă o punte între controlere și repository-urile care accesează datele. Folderul conține servicii precum AccountService.cs, AchievementService.cs, CalendarService.cs, EmailService.cs, EventService.cs, ExpenseService.cs, FinanceService.cs, goalService.cs, ReportService.cs, TokenService.cs, UserService.cs

De asemenea tot aici regăsim interfețele atât pentru serviciile menționate mai sus cât și pentru repository-urile din stratul de infrastructură.

MindfulWallet.Core: Acest strat corespunde layer-ului de bază al aplicației și include definițiile esențiale pentru funcționarea acesteia, cum ar fi obiectele de transfer de date (DTOs), entitățile de bază, ajutoarele și modelele de date.

Obiectele de transfer de date (DTOs) sunt utilizate pentru a transfera informații între diferitele straturi ale aplicației sau între aplicații diferite. Acestea conțin doar date și nu au logică de afaceri (AccountDto.cs, AchievementDto.cs, AddFundsDto.cs, EventDto.cs etc).

Entitățile reprezintă obiectele de bază ale sistemului și sunt utilizate pentru a modela datele și comportamentele acestora. Entitățile Mindful Wallet cuprind Account.cs, Achievement.cs, Calendar.cs, Event.cs, Expense.cs, Finance.cs, Goal.cs, RefreshToken.cs, Report.cs, ResetToken.cs și nu în ultimul rând, User.cs.

Ajutoarele (Helpers) conțin metode utilitare care sunt folosite pentru a realiza sarcini comune în aplicație, cum ar fi hashing-ul parolelor sau generarea de șabloane de email.

MindfulWallet.Infrastructure: Stratul conține implementările concrete pentru accesarea datelor și interacțiunea cu baza de date. Acesta este responsabil pentru gestionarea contextului bazei de date, migrațiilor și a repository-urilor care manipulează datele efective.

Cuprinde contextul bazei de date utilizat de Entity Framework pentru a interacționa cu baza de date: AppDbContext.cs, un folder ce conține migrările generate de Entity Framework pentru a gestiona modificările structurii bazei de date și un folder de repository-uri care implementează logica de acces la date pentru fiecare entitate. Repository-urile oferă un mod de a abstractiza interacțiunea cu baza de date și de a realiza operațiuni CRUD

Detalii ale Implementării

Fiecare strat are un rol specific și bine definit, ceea ce face ca fiecare parte a aplicației să fie mai ușor de gestionat și de testat.

Interfețele din MindfulWallet.Application permit decuplarea implementărilor concrete din MindfulWallet.Infrastructure. De exemplu, **IUserRepository** este definit în Application, iar implementarea **UserRepository** este în Infrastructure. Această decuplare asigură flexibilitate și posibilitatea de a schimba implementările fără a afecta logica de business.

Separarea logicii de business de infrastructură facilitează unit testing deoarece logica poate fi testată independent de baza de date sau alte servicii externe. Testele unitare pot fi

scrise pentru fiecare strat în parte, asigurând astfel o acoperire completă și identificarea rapidă a eventualelor erori.

Structura modulară permite adăugarea ușoară de noi funcționalități sau modificarea celor existente fără a afecta alte părți ale aplicației. Această abordare modulară permite extinderea aplicației în mod incremental și să integrarea de noi servicii sau tehnologii cu efort minim.

În concluzie, implementarea Clean Architecture în proiectul MindfulWalletAPI asigură o separare clară a responsabilităților și o decuplare a diferitelor părți ale aplicației, facilitând astfel mentenabilitatea, testabilitatea și scalabilitatea. Aceasta este o practică excelentă care contribuie la robustitatea și flexibilitatea aplicației.

3.1.2 Diagrama de clase si funcții

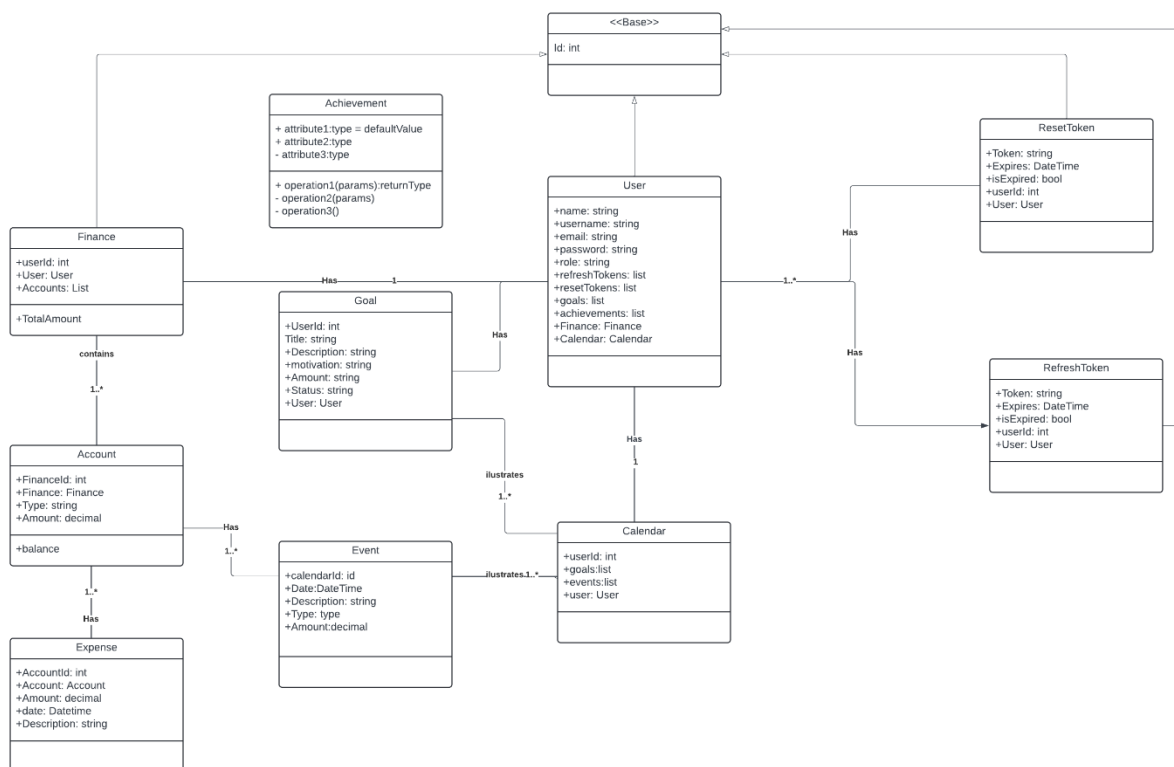


Figura 3 1

Diagrama de clasă a aplicației Mindful Wallet (Figura 3 1) reprezintă o structură complexă de entități și relații, fiecare având un rol specific în gestionarea resurselor financiare și a obiectivelor utilizatorilor. Fiecare clasă extinde BaseEntity, care adaugă automat un câmp Id unic pentru fiecare instanță, asigurând identificarea unică a fiecărei entități în baza de date.

Observatie – această moștenire nu a fost reprezentată pentru fiecare clasă pentru a nu supraîncărca diagrama.

Descrierea claselor

Clasa *User* este centrală în această diagramă deoarece întreaga aplicație se dezvoltă în jurul userului. Ea include proprietăți precum *Name*, *UserName*, *Email*, *Password* și *Role*, care sunt esențiale pentru gestionarea informațiilor de bază ale utilizatorului și pentru autentificare. Proprietatea *RefreshTokens* și *ResetTokens* sunt colecții care stochează tokenurile de reîmprospătare și resetare a parolei, asigurând securitatea și gestionarea sesiunilor utilizatorilor. Fiecare utilizator are asociat un obiect *Finance* și un *Calendar*, gestionând informațiile financiare și evenimentele personale. De asemenea, utilizatorul poate avea mai multe obiective (*Goals*) și realizări (*Achievements*), urmărind progresul și realizările financiare.

Clasa *Finance* este responsabilă pentru gestionarea conturilor financiare ale utilizatorului. Proprietățile sale includ *UserId*, care asociază acest obiect cu un utilizator specific, și *TotalAmount*, care reprezintă suma totală a fondurilor disponibile în toate conturile utilizatorului. Lista *Accounts* stochează toate conturile financiare ale utilizatorului, fiecare cont fiind reprezentat de clasa *Account*. Fiecare *cont* include proprietăți precum *FinanceId*, care îl asociază cu obiectul *Finance*, *Type*, care specifică tipul contului (de exemplu, economii, curent), și *Amount*, care reprezintă suma de bani disponibilă în acel cont. Cele 2 clase contin si functii ce permit accesarea totalului acumulat în cont și, respectiv a totalului din toate conturile utilizatorului

Clasa *Expense* gestionează cheltuielile efectuate din conturile utilizatorului. Proprietățile sale includ *AccountId*, care leagă cheltuiala de un cont specific, *Amount*, care reprezintă suma cheltuită, *Date*, care indică data efectuării cheltuielii, și *Description*, care oferă detalii despre natura cheltuielii. Aceste informații sunt cruciale pentru urmărirea cheltuielilor și gestionarea eficientă a fondurilor.

Clasa *Goal* permite utilizatorilor să stabilească și să urmărească obiective financiare. Proprietățile acestei clase includ *UserId*, asociind obiectivul cu un utilizator, *Title*, *Description*, *Motivation*, care oferă detalii și motivația obiectivului, *Amount*, care reprezintă suma de bani necesară pentru atingerea obiectivului, și *Status*, care indică progresul

obiectivului (de exemplu, în desfășurare, completat). Aceste proprietăți ajută utilizatorii să își planifice și să își atingă obiectivele financiare.

Clasa *Achievement* este utilizată pentru a urmări realizările utilizatorilor. Proprietățile sale includ *UserId*, *Title*, *Description*, *ImageUrl* și *DateAchieved*, care permit stocarea detaliilor complete ale realizărilor. Proprietatea *ImageUrl* este utilizată pentru a stoca adresa URL a unei imagini reprezentative pentru realizare. Imaginea poate fi un simbol sau o ilustrație care reflectă natura succesului, contribuind la vizualizarea și recunoașterea realizării.

Relația cu clasa *User* este gestionată prin proprietatea *User*, care este o referință către utilizatorul asociat realizării. Această relație este definită folosind atributul *ForeignKey*, indicând că *UserId* este cheie străină care leagă realizarea de utilizator.

Clasa *Calendar* gestionează evenimentele utilizatorului. Proprietățile sale includ *UserId*, care leagă calendarul de un utilizator, și listele *Goals* și *Events*, care stochează obiectivele și evenimentele utilizatorului. Acest lucru permite utilizatorilor să își planifice activitățile și să urmărească evenimentele importante legate de finanțele lor.

Clasa *Event* este utilizată pentru a reprezenta evenimente specifice în calendarul utilizatorului. Proprietățile includ *CalendarId*, *Date*, *Description*, *Type*, și *Amount*, care oferă detalii despre eveniment, inclusiv data, descrierea, tipul (de exemplu, depozit, retragere) și suma implicată. Aceste evenimente ajută la urmărirea acțiunilor financiare și la menținerea unei evidențe clare a tranzacțiilor.

Funcțiile:

Operațiunile în backend-ul aplicației includ înregistrarea, logarea, autentificarea și gestionarea resurselor utilizatorului. Procesul de înregistrare implică validarea informațiilor introduse, hashuirea parolei, crearea unui obiect *Finance* și *Calendar* pentru utilizatorul nou. Logarea validează credențialele utilizatorului și generează un JWT și un refresh token, care sunt trimise în răspuns. Autentificarea prin refresh token implică generarea unui nou JWT folosind un refresh token valid, validarea token-ului și generarea unor noi token-uri.

Utilizatorii pot adăuga conturi noi prin endpoint-ul */api/Account/{userId}*, care creează un cont nou pentru un utilizator specificat și îl adaugă în lista de conturi a obiectului *Finance* asociat utilizatorului. Ștergerea unui cont se realizează prin endpoint-ul */api/Account/{accountId}*, ștergând contul specificat și toate evenimentele asociate acestuia.

Utilizatorii pot adăuga cheltuieli noi prin endpoint-ul `/api/Expense`, creând o nouă cheltuială în contul specificat și adăugând un eveniment corespunzător în calendar. Generarea rapoartelor se realizează prin endpoint-ul `/api/Report`, creând și actualizând rapoartele financiare pe baza datelor din conturile și cheltuielile utilizatorului.

Un aspect important al funcționalității backend-ului este generarea automată a realizărilor (Achievements). Atunci când utilizatorul își atinge un obiectiv financiar, sistemul verifică dacă suma economisită într-un cont de economii îndeplinește cerințele unui obiectiv setat. Dacă da, obiectivul este marcat ca finalizat și se creează o nouă realizare. Acest proces automatizat asigură că utilizatorii sunt mereu la curent cu progresul lor, fără a necesita intervenția manuală.

3.1.3 Frontend

În cadrul proiectului Angular, a fost aplicată arhitectura bazată pe componente pentru a asigura o organizare clară a codului în module, componente, servicii și alte elemente structurale. Această abordare promovează separarea responsabilităților, reutilizabilitatea și mentenabilitatea aplicației.

Structura Proiectului:

Components este directorul principal care conține toate componentele aplicației. Fiecare componentă are propriile fișiere pentru logică, template și stiluri, reflectând arhitectura bazată pe componente. Aplicația conține componenta calendar, ce cuprinde logica și afișarea evenimentelor din calendar, componenta expenses pentru gestionarea cheltuielilor, ce include afișarea și manipularea datelor legate de cheltuieli, goals pentru gestionarea obiectivelor financiare ale utilizatorului, home - componenta pentru pagina de start a aplicației etc.

Gărzile (guards) sunt utilizate pentru a proteja rutele și a controla accesul la diferite părți ale aplicației. Aici este de menționat *auth.guard.ts*, garda pentru autentificare, care verifică dacă un utilizator este autentificat înainte de a permite accesul la anumite rute.

Directorul helpers conține funcții și utilitare care sunt folosite în mai multe locuri din aplicație, precum *validateForm.ts* ce se ocupa de validarea formularelor de tip login si autentificare.

Interceptors conține interceptori HTTP, care interceptează și manipulează cererile și răspunsurile HTTP. Aici putem menționa *token.interceptor.ts*, interceptorul pentru adăugarea token-urilor de autentificare la cererile HTTP.

Directorul *models* conține definițiile de tipuri și interfețe utilizate în aplicație (*event.model.ts*, *goal.model.ts*, *reset-password.model.ts*, *token-api.model.ts*).

Nu în ultimul rând, directorul *Services* conține serviciile care sunt folosite pentru a partaja date și logică între componente.

Câteva servicii cu funcționalități importante: *auth.service.ts* se ocupă cu gestionarea autentificării și autorizării utilizatorilor. Acest serviciu include metode pentru login, logout, și verificarea token-urilor de acces, *event.service.ts* se ocupa cu Gestionarea evenimentelor și include metode pentru crearea, actualizarea, ștergerea și recuperarea evenimentelor, iar *finance.service.ts* are în vedere gestionarea finanțelor personale, incluzând metode pentru adăugarea, actualizarea, ștergerea și recuperarea tranzacțiilor financiare.

În rădăcina directorului *app*, se regăsesc fișierele care configurează aplicația și rutele acestuia: *app.component.ts(.html, .css)* este componenta principală a aplicației, care servește ca punct de intrare pentru Ui; *app.routes.ts* descrie configurația rutelor aplicației, iar *app.config.ts* conține configurațiile generale ale aplicației.

Concluzionând, prin organizarea codului în module și componente independente, fiecare cu responsabilități clar definite, s-a implementat cu succes arhitectura bazată pe componente în proiectul Angular. Aceasta promovează o bună separare a responsabilităților, făcând codul mai ușor de întreținut, testat și extins pe măsură ce cerințele aplicației evoluează. De asemenea, structura modulară permite adăugarea ușoară de noi funcționalități sau modificarea celor existente fără a afecta alte părți ale aplicației. Aceasta abordare asigură robustitatea și flexibilitatea aplicației, pregătind-o pentru cerințele viitoare și adaptările necesare în domeniul tehnologic.

3.1.4 Diagrama de flux

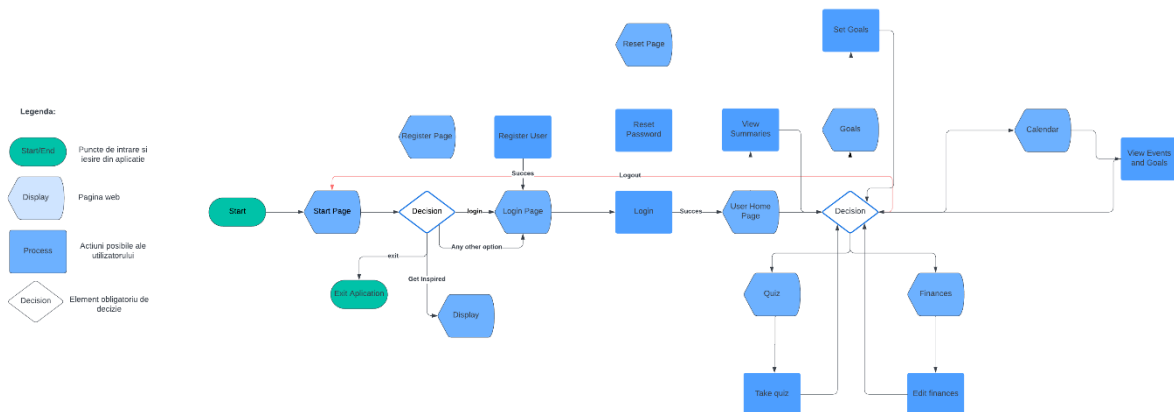


Figura 3 2

Diagrama de flux prezentată pentru aplicația Mindful Wallet ilustrează diferitele căi și decizii pe care utilizatorii le pot lua în cadrul aplicației, oferind o imagine clară a funcționalităților disponibile:

Fluxul începe cu nodul Start, care duce utilizatorul la Start Page. Aceasta este pagina principală de pornire a aplicației, care prezintă pe scurt punctele forte ale aplicației și scopul ei. Aici utilizatorul poate alege diferite opțiuni pentru a continua.

De la Start Page, așa cum am menționat, utilizatorul ajunge la un punct de Decizie. Aici, utilizatorul poate alege să se înregistreze, să se logheze, să vizualizeze paginile de Instagram sau YouTube ale aplicației sau să iasă din aplicație. Dacă utilizatorul alege să iasă din aplicație, fluxul se încheie la nodul Exit Application.

Dacă utilizatorul decide să se înregistreze, acesta este direcționat către Register Page. În această pagină, utilizatorul introduce informațiile necesare pentru a crea un cont nou. După completarea formularului de înregistrare, acțiunea Register User are loc. Dacă înregistrarea este reușită, utilizatorul este direcționat către Login Page. În caz de eșec, utilizatorul poate încerca din nou.

Utilizatorul poate alege să se logheze, fiind direcționat către Login Page. După introducerea corectă a informațiilor de autentificare, acțiunea Login are loc. Dacă autentificarea este reușită, utilizatorul este direcționat către User Home Page. Sau, în cazul în care utilizatorul a uitat parola, opțiunea Forgot password din Login Page îl direcționează către Reset Page. Aici, utilizatorul poate introduce adresa de email pentru a reseta parola. După acțiunea Reset Password, utilizatorul poate încerca din nou să se logheze.

După logarea cu succes, utilizatorul ajunge la User Home Page. Aceasta este pagina principală a utilizatorului, unde sunt afișate cateva informații ce rezumă activitatea utilizatorului in cadrul aplicației. Tot acum, utilizatorul poate decide să acceseze diverse funcționalități ale aplicației, precum Finances, Goals, Calendar sau poate alege să completeze un chestionar care îi va răspunde cu succes la întrebarea „Ar trebui sa cumpar prosusul X?”.

Utilizatorul poate accesa Goals pentru a seta și gestiona obiectivele financiare. Acțiunea Set Goals permite utilizatorului să definească noi obiective, care sunt apoi vizibile și gestionabile atat din pagina Goals, cat si in Calendar.

Pagina Calendar permite vizualizarea evenimentelor și obiectivelor planificate.

Utilizatorul poate edita informațiile financiare prin accesarea paginii Finances. Acțiunea Edit Finances permite modificarea detaliilor conturilor financiare, gestionarea cheltuielilor și actualizarea sumelor disponibile.

Utilizatorul poate alege să se delogheze din aplicație în orice moment. Acțiunea Logout duce înapoi la pagina de Start, finalizând sesiunea curentă a utilizatorului.

3.2 Baza de date

3.2.1 Abordarea code first

Pentru aplicația Mindful Wallet, am utilizat abordarea Code First din Entity Framework (EF) Core pentru a crea și gestiona baza de date. Această metodă permite definirea structurii bazei de date folosind clase C#, iar EF Core generează schema bazei de date pe baza acestor definiții. Acest proces oferă un control complet asupra modelului de date și simplifică gestionarea și evoluția bazei de date pe măsură ce aplicația se dezvoltă.

În abordarea Code First, modelul de date este definit prin crearea de clase C# care reprezintă tabelele din baza de date. Fiecare clasă corespunde unei tabele, iar proprietățile clasei corespund coloanelor tabelului. De exemplu, pentru a reprezenta utilizatorii din sistem, se definește o clasă care include proprietăți precum ID-ul utilizatorului, numele, adresa de email și parola.

Contextul (*DbContext*) este configurat pentru a gestiona accesul la baza de date. Acesta definește proprietăți care corespund fiecărui model, reprezentând tabelele din baza de date. Contextul este responsabil pentru interacțiunile cu baza de date, cum ar fi adăugarea, actualizarea și ștergerea înregistrărilor.

Conexiunea la baza de date este specificată în fișierul de configurare al aplicației, *appsettings.json*, unde se definește șirul de conexiune. Acest șir de conexiune este utilizat în cadrul configurării serviciilor aplicației pentru a stabili legătura cu baza de date specificată.

Pentru a genera schema bazei de date pe baza modelului definit, se folosesc migrations. O migrare este o serie de modificări incrementale aplicate bazei de date pentru a o aduce la starea dorită. Migration-urile sunt generate automat de EF Core pe baza modificărilor aduse modelului de date și sunt aplicate bazei de date pentru a sincroniza structura acestuia cu modelul definit în cod.¹³

Pentru mai multe informații legate de Code-first approach, vezi [9] din bibliografie.

Pe măsură ce aplicația evoluează, modelul de date poate necesita modificări. Abordarea Code First permite gestionarea ușoară a acestor modificări prin adăugarea de noi migrations care descriu schimbările necesare.

¹³ Sahu, A., 2023. Implement Entity Framework A Code First Approach in .Net 8 API. Disponibil la: <https://medium.com/@codewithhanksahu/implement-entity-framework-a-code-first-approach-in-net-8-api-80b06d219373> [Accesat 04 June 2024].

3.2.2 Diagrama Entity-Relationship

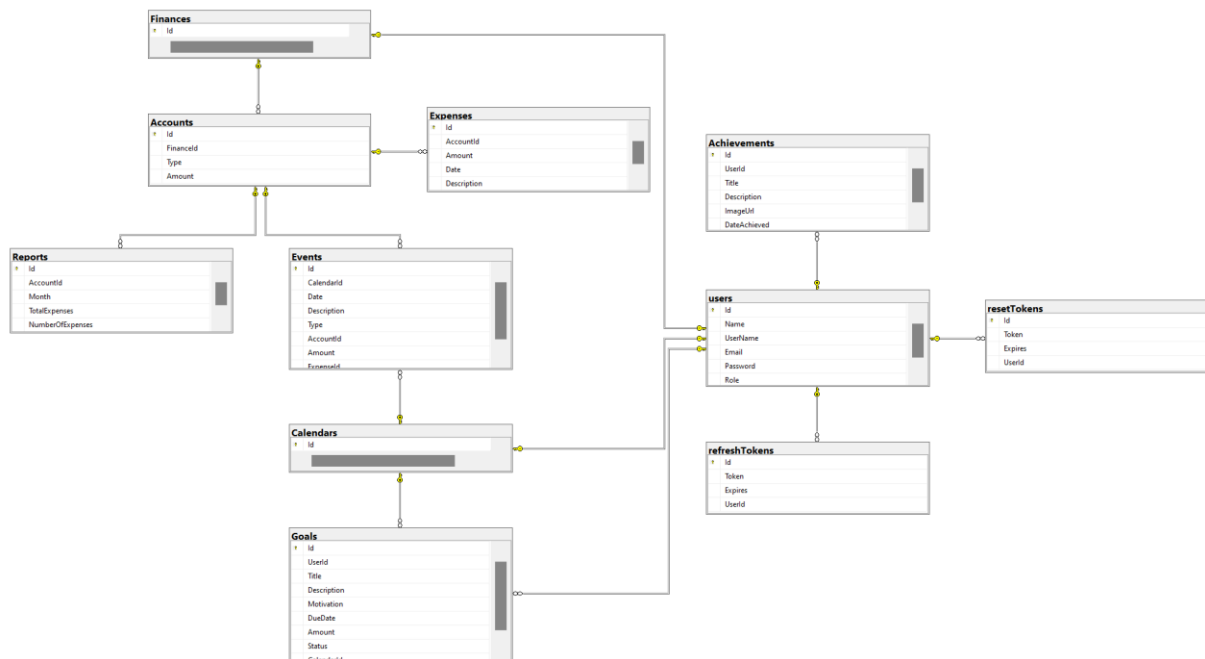


Figura 3 3

3.2.3 Descrierea entităților și a relațiilor

Mindful Wallet utilizează un model de date complex și bine structurat pentru a gestiona eficient finanțele personale ale utilizatorilor săi. Acest model este alcătuit din multiple entități interconectate, fiecare având un rol specific în cadrul aplicației. În cele ce urmează, voi descrie fiecare entitate și relațiile dintre acestea.

Entitatea *Users* reprezintă informațiile esențiale despre utilizatorii aplicației. Aceasta include câmpuri precum Name, Username, Email, Password și Role. Utilizatorii au relații de tip 1-1 cu entitățile *Finance* și *Calendar* și pot deține mai multe token-uri de resetare și de reîmprospătare.

Finance stochează informațiile financiare ale unui utilizator. Aceasta conține o colecție de conturi financiare (entitatea *Accounts*), permițând gestionarea centralizată a tuturor activelor financiare ale utilizatorului.

Entitatea *Accounts* reprezintă conturile financiare ale utilizatorilor. Fiecare cont este asociat cu un *Finance* și poate conține multiple cheltuieli (*Expenses*) și rapoarte (*Reports*). Conturile

au câmpurile *Type* și *Amount*, iar balanța contului este calculată ca diferența dintre suma totală și cheltuielile asociate.

Entitatea *Expenses* stochează informațiile despre cheltuielile efectuate dintr-un cont. Fiecare cheltuială este legată de un *Account* și include câmpuri precum *Amount*, *Date* și *Description*. Aceasta permite urmărirea detaliată a cheltuielilor utilizatorului.

Reports generează rapoarte lunare ale cheltuielilor pentru un cont specific. Fiecare raport este asociat cu un *Account* și conține informații despre totalul cheltuielilor și numărul acestora pentru o lună specifică.

Entitatea *Events* stochează informațiile despre evenimentele financiare, cum ar fi depozitele și retragerile. Fiecare eveniment este asociat cu un *Calendar*.

Entitatea *Calendars* gestionează calendarele utilizatorilor, incluzând evenimente (*Events*) și obiective (*Goals*). Fiecare calendar este legat de un *User*, oferind o vedere de ansamblu asupra evenimentelor și obiectivelor financiare ale utilizatorului.

Goals stochează informațiile despre obiectivele financiare ale utilizatorilor. Fiecare obiectiv este asociat cu un *User* și include câmpuri precum *Title*, *Description*, *Motivation*, *DueDate*, *Amount* și *Status*.

Entitatea *Achievements* reprezintă realizările financiare ale utilizatorilor. Fiecare realizare este asociată cu un *User* și conține informații precum *Title*, *Description*, *ImageUrl* și *DateAchieved*.

Entitățile *ResetTokens* și *RefreshTokens* stochează token-urile utilizate pentru resetarea și reîmprospătarea parolelor. Fiecare token este legat de un *User* și include câmpuri precum *Token* și *Expires*.

3.3 Modul de funcționare al aplicației

3.3.1 User case diagram

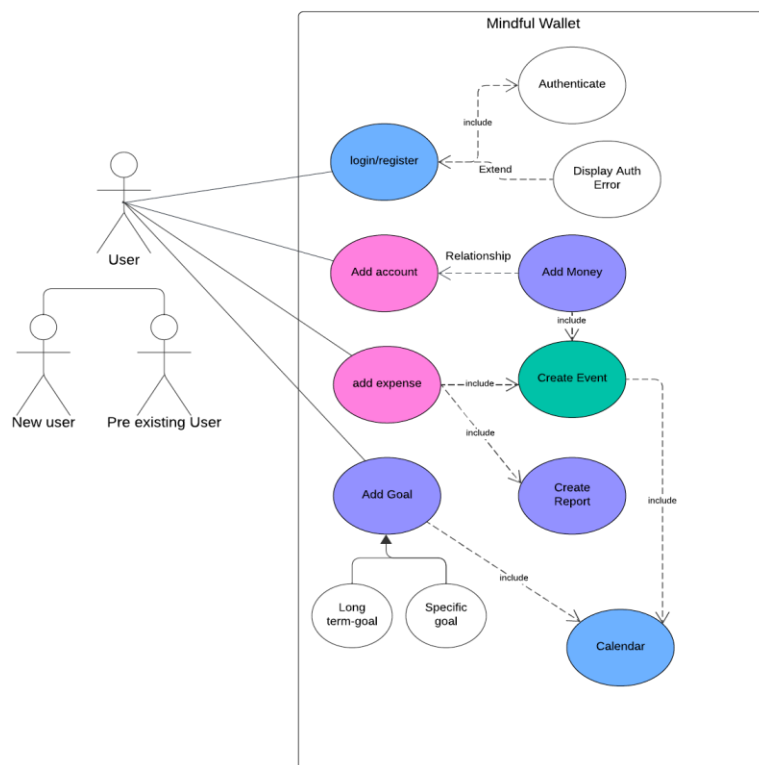


Figura 3 4

Diagrama de cazuri de utilizare ilustrează interacțiunile dintre utilizatori și funcționalitățile principale ale aplicației, evidențiind diverse acțiuni și relații dintre ele.

Utilizatorul general, reprezentat prin actorul User, poate efectua mai multe acțiuni în cadrul aplicației. Există două subtipuri de utilizatori, și anume New User și Pre-existing User, care sunt ilustrate pentru a arăta diferențele dintre acțiunile disponibile pentru utilizatorii noi și cei existenți.

Utilizatorii pot începe prin a se autentifica sau a se înregistra, reprezentate prin cazul de utilizare login/register. Acesta este primul pas esențial pentru accesarea aplicației. Dacă autentificarea eșuează, este afișată o eroare de autentificare, indicată prin cazul de utilizare Display Auth Error.

După autentificare, utilizatorii pot adăuga un cont nou folosind cazul de utilizare Add account. Această acțiune este esențială pentru gestionarea resurselor financiare personale. După ce au adăugat un cont, utilizatorii pot adăuga bani în contul respectiv, acțiune indicată prin cazul de utilizare Add Money. Adăugarea de bani generează un eveniment, reprezentat de

cazul de utilizare Create Event, care este important pentru menținerea unei evidențe clare a tuturor tranzacțiilor financiare.

Utilizatorii pot de asemenea să adauge cheltuieli, acțiune ilustrată prin cazul de utilizare add expense. Aceasta le permite să urmărească cheltuielile efectuate din conturile lor, oferind o imagine completă a stării financiare. Pentru a-și planifica și gestiona mai bine resursele, utilizatorii pot stabili obiective financiare prin cazul de utilizare Add Goal. Obiectivele financiare pot fi de lungă durată sau specifice, așa cum este reprezentat de subcazurile Long term-goal și Specific goal.

Un alt aspect important al aplicației este generarea de rapoarte, ilustrată prin cazul de utilizare Create Report. Aceasta funcționalitate crează rapoarte financiare detaliate, bazate pe tranzacțiile utilizatorilor. Rapoartele ajută la analizarea și evaluarea stării financiare, oferind utilizatorilor o înțelegere clară a progresului lor financiar.

În plus, aplicația include un calendar, reprezentat prin cazul de utilizare Calendar. Calendarul permite utilizatorilor să vizualizeze și să gestioneze evenimentele și obiectivele financiare. Aceasta funcționalitate ajută la planificarea și organizarea activităților financiare, oferind o imagine de ansamblu asupra tuturor evenimentelor importante.

Capacitatea de a stabili și urmări obiective financiare este o altă funcționalitate cheie a aplicației (Add Goal). Acest proces permite utilizatorilor să își definească obiectivele financiare, fie că sunt de lungă durată sau specifice, așa cum se vede în subcazurile Long term-goal și Specific goal. Stabilirea obiectivelor ajută utilizatorii să planifice și să urmărească progresul lor financiar, oferindu-le motivație și o direcție clară.

3.3.2 Diagrama de interacțiuni user - frontend - backend

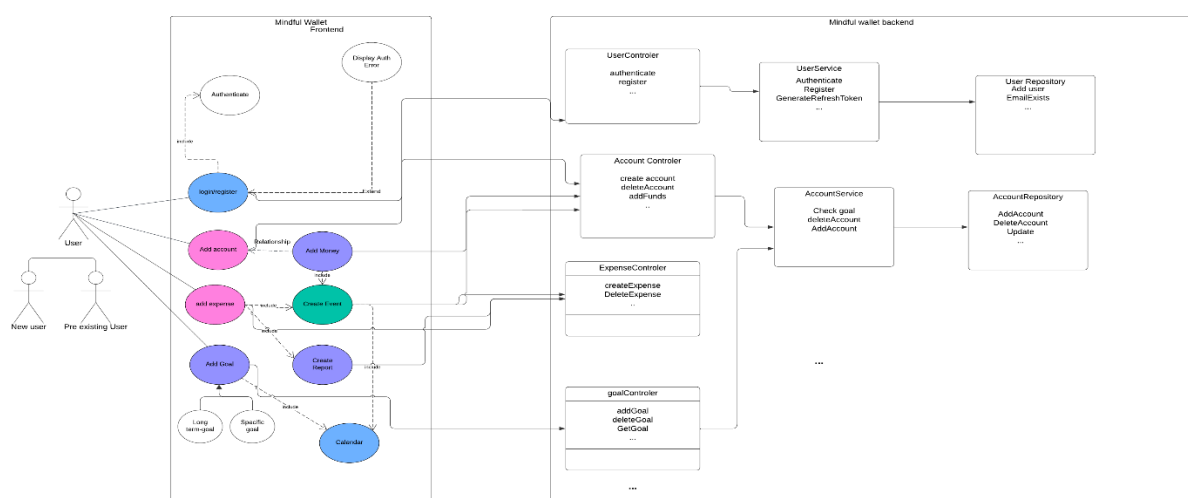


Figura 3 5

Diagrama de interacțiuni prezentată ilustrează logica generală a aplicației Mindful Wallet, evidențiind interacțiunile utilizatorului cu frontend-ul și backend-ul. Fiecare interacțiune a utilizatorului declanșează un apel către API-urile controlerelor, care, la rândul lor, apelează serviciile și repozițiile aferente.

De exemplu, în cazul autentificării unui user, acesta introduce datele de autentificare (email și parolă) pe pagina de login din frontend. Aceste date sunt apoi trimise printr-o cerere HTTP POST către endpoint-ul `/api/UserController/authenticate` din backend. Controllerul `UserController` primește cererea de autentificare de la frontend. Acesta este responsabil pentru validarea inițială a datelor primite. Dacă datele sunt valide, controllerul invocă metoda `AuthenticateAsync` din serviciul `IUserService`. În cazul în care datele nu sunt valide, controllerul returnează un mesaj de eroare adecvat către frontend.

Serviciul `UserService` implementează logica principală de autentificare. Această metodă face mai întâi apel la `UserRepository` pentru a verifica dacă există un utilizator cu adresa de email furnizată. Dacă utilizatorul nu este găsit sau parola nu este corectă, serviciul returnează null, semnalând eșecul autentificării.

Dacă utilizatorul este găsit și parola este corectă, serviciul continuă cu generarea unui JWT și a unui refresh token. Pentru generarea JWT-ului, serviciul face apel la `TokenService`, care creează token-ul utilizând informațiile de autentificare ale utilizatorului și cheile de securitate configurate.

Serviciul `UserService` generează apoi un refresh token, care este un token unic și securizat folosit pentru a obține un nou JWT fără a reintroduce credențialele utilizatorului. Acest refresh token este stocat în baza de date, asociat cu utilizatorul curent, pentru a permite reînnoirea token-urilor de acces.

După ce JWT-ul și refresh token-ul sunt generate, `UserService` le returnează către `UserController`, care le trimite apoi în răspunsul către frontend, indicând succesul autentificării.

Astfel interacțiunea urmărește schema următoare Frontend -> Controller -> Services-> Repository ->services->Controller -> Frontend.

Pentru a nu îngreuna înțelegerea diagramei, am omis adăugarea tuturor elementelor prezente în backend și frontend, însă vom analiza, în detaliu, pe rând, aceste interacțiuni User – Frontend - Backend pe măsură ce descriem implementarea paginilor din aplicație.

3.4 Implementarea Paginilor

3.4.1 Pagina de login.

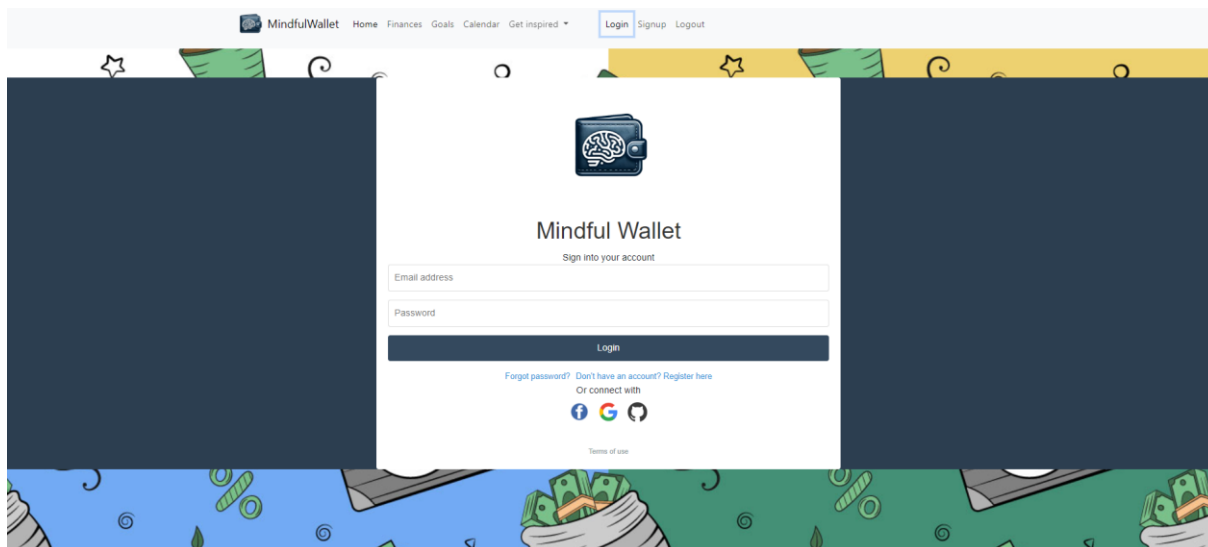


Figura 3 6

Pagina de autentificare (figura 3.6) prezintă un design clasic, punând accent pe funcționalitate. Pagina este centrată pe un formular simplu, în care utilizatorul trebuie să introducă credențialele pentru a se putea loga în aplicație. Formularul utilizează tehnici de validare care asigură faptul ca utilizatorul introduce datele în formatul corect în câmpuri.

Linkurile pentru recuperarea parolei și înregistrarea utilizatorilor noi sunt plasate convenabil sub formular, facilitând navigarea ușoară.

Interacțiunile User-Frontend-Backend:

Componenta LoginComponent este responsabilă pentru gestionarea logicii paginii de login. Aceasta utilizează servicii pentru a manipula datele necesare autentificării utilizatorului. AuthService gestionează cererile legate de autentificare, cum ar fi verificarea credențialelor și stocarea tokenului de autentificare.

Procesul de **autentificare** începe cu utilizatorul care introduce datele de autentificare pe pagina de login din frontend. Aceste date sunt apoi trimise printr-o cerere HTTP POST către endpoint-ul `/api/UserController/authenticate` din backend. Procesul continuă așa cum a fost descris anterior în subcapitolul 3.3.2.

Pe lângă procesul de autentificare, pagina de login include și funcționalitatea de **resetare a parolei**. Utilizatorul poate accesa această funcționalitate printr-un link plasat sub formularul de autentificare care deschide un modal specific:

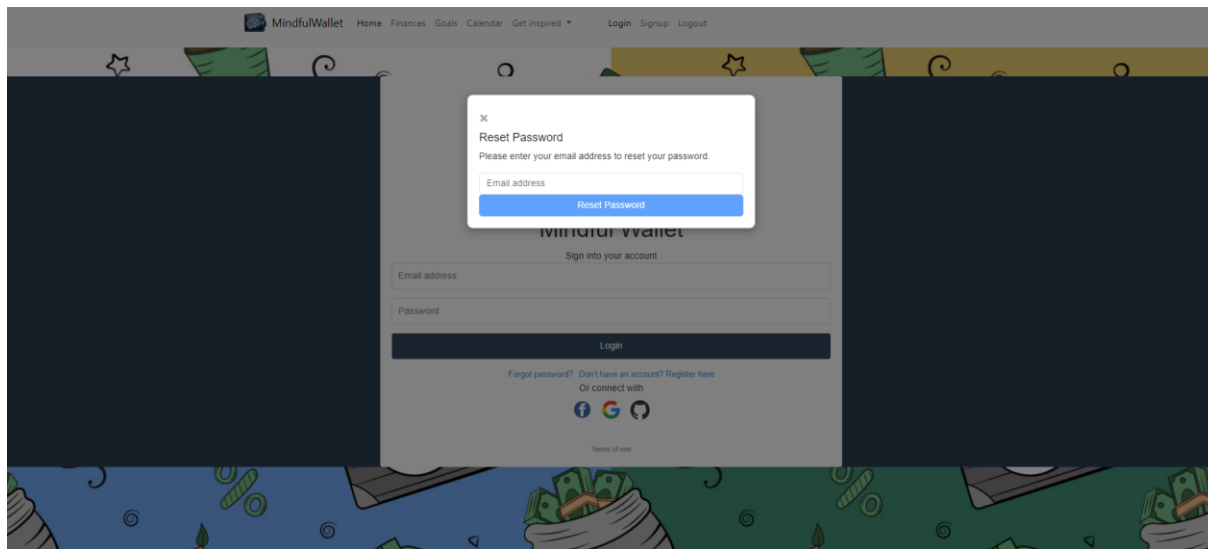


Figura 3 7

Modalul pentru resetarea parolei (Figura 3 7) solicită utilizatorului să introducă adresa de email. Adresa de email este validată în timp real pentru a asigura că este introdusă corect. Dacă adresa de email este validă, utilizatorul poate trimite cererea de resetare a parolei.

Modalul este o adiție utilă ce ne salvează de necesitatea creării unei întregi noi pagini html; se adaugă direct în pagina în care va apărea, în afara componentei <body>.

```
<!-- Modal -->
<div class="modal fade" id="forgotPasswordModal" tabindex="-1" aria-
labelledby="forgotPasswordModalLabel" aria-hidden="true">
  <div class="modal-dialog">
    <div class="modal-content">
      <div class="modal-header">
        <h5 class="modal-title" id="forgotPasswordModalLabel">Reset Password</h5>
        <button type="button" class="btn-close" data-bs-dismiss="modal" aria-label="Close"></button>
      </div>
      <div class="modal-body">
        <p>Please enter your email address to reset your password.</p>
        <input [(ngModel)]="resetPasswordEmail" (ngModelChange)="checkValidEmail($event)"
type="email" id="reset-email" class="form-control" placeholder="Email address" required>
        <small *ngIf="!isValidEmail && resetPasswordEmail" class="text-danger">
          *Please enter a valid email address
        </small>
      </div>
    </div>
  </div>
</div>
```

```

</div>
<div class="modal-footer">
    <button type="button" id="closeModalbtn" class="btn btn-secondary" data-bs-
dismiss="modal">Close</button>
    <button type="button" (click)="sendResetEmail()" class="btn btn-primary"
[disabled]="!isValidEmail">Reset Password</button>
</div>
</div>
</div>
</div>

```

Pentru gestionarea cererilor de resetare a parolei, am creat un serviciu dedicat. Acest serviciu are metode pentru trimiterea email-ului de resetare și pentru resetarea efectivă a parolei. Metoda de trimitere a email-ului trimite o cerere POST către server, endpoint-ul `/api/UserController/send-reset-email`, indicând adresa de email a utilizatorului. Metoda de resetare a parolei trimite o cerere POST cu obiectul care conține noua parolă și alte informații necesare pentru resetare.

Un helper *EmailBody* este folosit pentru a genera conținutul HTML al email-ului de resetare a parolei (Figura 3 8), care include un link pentru resetarea parolei. Astfel, emailul-ul de resetare a parolei are un aspect mai placut pentru utilizator.

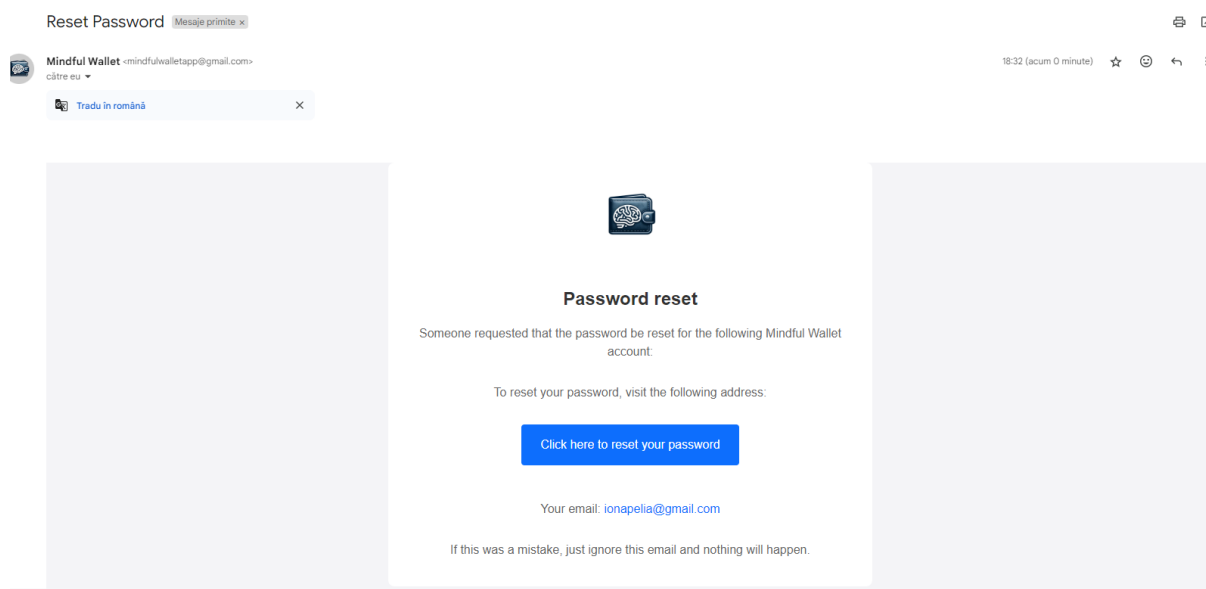


Figura 3 8

Backend-ul primește cererea și generează un token de resetare a parolei, pe care îl trimite la adresa de email furnizată. Utilizatorul poate folosi acest link din email pentru a accesa pagina de resetare a parolei (Figura 3.9), unde poate introduce o nouă parolă. Procesul de resetare a parolei implică validarea token-ului și actualizarea parolei utilizatorului în baza de date, folosind metodele din UserService și UserRepository.

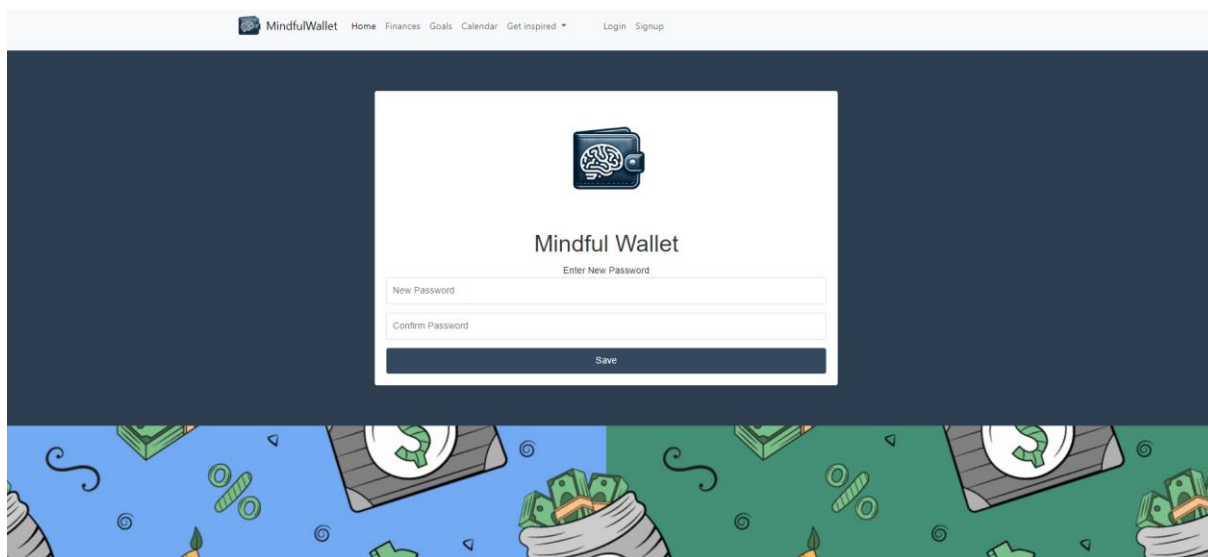


Figura 3.9

Formularul de resetare a parolei include validări pentru a se asigura că parola îndeplinește cerințele de securitate, cum ar fi lungimea minimă și prezența caracterelor speciale, cifrelor și literelor majuscule și minuscule.

După ce utilizatorii introduc și confirmă noua parolă, un mesaj de succes le indică faptul că parola a fost resetată cu succes și sunt direcționați către pagina principală a aplicației.

Securitatea Datelor

Pentru asigurarea securității datelor utilizatorilor, Mindful Wallet implementează mai multe măsuri de securitate, precum Autentificarea JWT, folosirea tokenurilor de reîmprospatare și hasuirea parolelor.

- Autentificarea JWT (JSON Web Token):

TokenService generează token-uri JWT folosind JwtSecurityTokenHandler și o cheie secretă specificată în configurarea aplicației.

Cheia (Secretul) este stocat în User secrets ceea ce asigură securitatea conturilor utilizatorilor. Astfel acesta nu poate fi vizualizat în codul sursă.

Token-ul JWT conține revendicări (claims) precum rolul și numele de utilizator și are o durată de valabilitate configurabilă. Autentificarea JWT permite serverului să verifice identitatea utilizatorului fără a stoca sesiuni pe server.

- Token-uri de Reîmprospătare:

Token-urile de reîmprospătare sunt generate folosind `RandomNumberGenerator` și sunt stocate în baza de date pentru fiecare utilizator.

Aceste token-uri permit utilizatorului să obțină un nou token JWT fără a se reautentifica, asigurând o experiență de utilizare fluidă.

Metoda `RefreshTokenAsync` din `UserService` validează și generează noi token-uri de reîmprospătare.

- Hashuirea Parolelor:

Parolele utilizatorilor sunt securizate folosind `PasswordHasher`, care utilizează algoritmul `Rfc2898DeriveBytes` pentru a genera un hash sigur și un salt unic pentru fiecare parolă.

La autentificare, parola introdusă este verificată comparând hash-ul generat cu hash-ul stocat în baza de date. Acest proces asigură că parolele nu sunt stocate în formă clară și sunt protejate împotriva atacurilor de tip brute force și dictionary.

Implementarea acestor măsuri de securitate asigură protecția datelor utilizatorilor și integritatea sistemului de autentificare al aplicației Mindful Wallet.

3.4.2 Pagina de signup.

The screenshot shows the 'Sign up' page for MindfulWallet. The page layout includes a top navigation bar with the logo and links for Home, Finances, Goals, Calendar, Get inspired, Login, and Signup. The main content area is divided into three sections. The left section is a decorative panel with a light blue background, featuring illustrations of a safe, a money bag, and coins. The right section is a decorative panel with a yellow and green background, featuring illustrations of a money bag, a briefcase, and coins. The central section contains the 'Sign up' form. The form has a title 'Sign up' and a list of items to track: Savings, Expenses, and Wishlist. Below the list are five input fields: 'Your Name', 'Your Username', 'Your Email', 'Password', and 'Repeat your password'. At the bottom of the form is a checkbox for 'I agree to all statements in [Terms of service](#)' and a 'Register' button.

Figura 3 10

Pagina de signup (Figura 3 10) continuă etosul de design utilizat până acum, având un layout asemanător cu pagina de login si folosind culori din palete similare.

Aceasta permite utilizatorilor noi să se înregistreze completând informații personale de bază, cum ar fi numele, emailul și parola, împreună cu confirmarea parolei pentru a preveni erorile de tastare. Formularul integrează de asemenea validarea reactivă, care verifică activ corectitudinea intrărilor, cum ar fi validarea formatului adresei de email și asigurarea că parola îndeplinește standardele de securitate necesare.

O caracteristică notabilă pe pagina de înregistrare este căsuța de selectare pentru acceptarea Termenilor de Serviciu, esențială pentru asigurarea înțelegerii utilizatorilor asupra drepturilor și obligațiilor lor.

Formularul incorporează validarea reactivă a formularului din Angular pentru a oferi o experiență de utilizare interactivă și receptivă. Această metodă permite rularea validărilor în mod asincron alături de introducerea utilizatorului, oferind feedback în timp real și îmbunătățind utilizabilitatea și accesibilitatea generală a formularelor.

Interacțiunile User-Frontend-Backend

Componenta *SignupComponent* este responsabilă pentru gestionarea logicii paginii de signup. Aceasta utilizează *AuthService* pentru a gestiona cererile legate de înregistrare și validarea datelor utilizatorilor.

În cazul înregistrării unui utilizator, acesta completează formularul de pe pagina de signup din frontend. Datele introduse de utilizator sunt trimise printr-o cerere HTTP POST către endpoint-ul */api/UserController/register* din backend. Controllerul *UserController* primește cererea de înregistrare de la frontend și este responsabil pentru validarea inițială a datelor primite. Dacă datele sunt valide, controllerul invocă metoda *RegisterUserAsync* din serviciul *IUserService*. În cazul în care datele nu sunt valide, controllerul returnează un mesaj de eroare adecvat către frontend.

Serviciul *UserService* implementează logica principală de înregistrare. Această metodă face mai întâi apel la *UserRepository* pentru a verifica dacă numele de utilizator sau adresa de email există deja în baza de date. Dacă există, serviciul returnează un mesaj de eroare specific, semnalând faptul că înregistrarea nu poate continua. Dacă numele de utilizator și adresa de email sunt unice, serviciul creează un nou obiect *User* cu datele utilizatorului.

3.4.3 Pagina Finanțe.

Pagina de finanțe din aplicația "Mindful Wallet" este concepută pentru a oferi utilizatorilor o interfață clară și intuitivă pentru gestionarea conturilor financiare, adăugarea de fonduri și cheltuieli, precum și vizualizarea rapoartelor financiare lunare. Voi descrie structura și funcționalitățile principale ale acestei pagini.

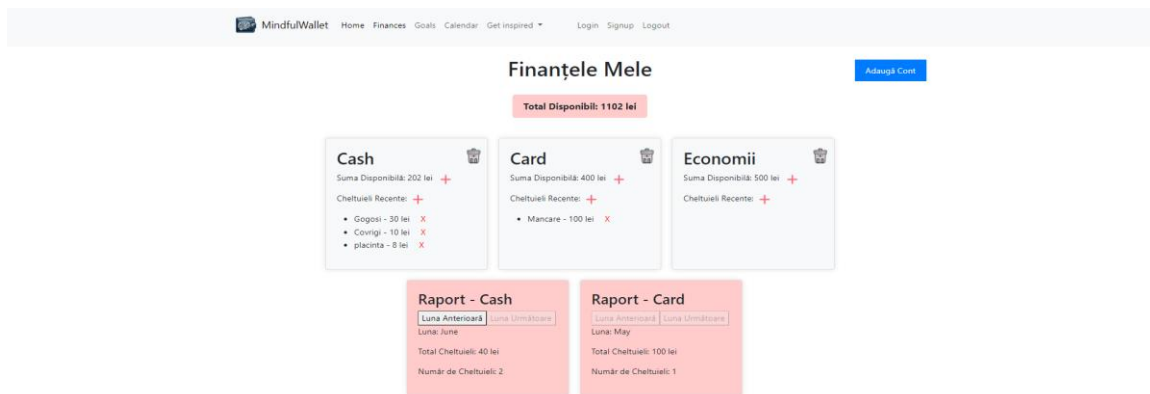


Figura 3 11

Pagina de finanțe utilizează un layout HTML simplu și eficient, împărțit în mai multe secțiuni:

Secțiunea de antet conține un titlu și un buton pentru adăugarea unui cont nou. Imediat sub antet, este afișată suma totală disponibilă în toate conturile utilizatorului.

Gridul de Conturi afișează o listă de carduri, câte unul pentru fiecare cont al utilizatorului. Fiecare card include informații despre tipul contului, suma disponibilă și un buton pentru adăugarea de fonduri sau cheltuieli. De asemenea, sunt afișate cheltuielile recente, cu opțiunea de a șterge o cheltuială sau contul.

Fiecare cont are o secțiune dedicată pentru rapoartele de cheltuieli lunare, care include navigarea între rapoartele lunilor anterioare și următoare.

Interacțiunile User-Frontend-Backend

Toate acțiunile de adăugare, ștergere și actualizare sunt gestionate prin metode definite în componenta `ExpensesComponent`, care se ocupă de manipularea formularului și de actualizarea datelor în UI.

1. Adăugarea unui cont nou:

Utilizatorul accesează pagina de finanțe și dorește să adauge un cont nou. Acesta completează formularul de pe frontend pentru adăugarea unui cont nou. Datele introduse de utilizator sunt trimise printr-o cerere HTTP POST către endpoint-ul `/api/AccountController/{userId}` din backend. Controllerul `AccountController` primește

cererea de creare a unui cont nou și este responsabil pentru validarea inițială a datelor primite. Dacă datele sunt valide, controllerul invocă metoda *AddAccountAsync* din serviciul *IAccountService*. În cazul în care datele nu sunt valide, controllerul returnează un mesaj de eroare adecvat către frontend.

Serviciul *AccountService* implementează logica principală de creare a unui cont nou. Metoda face mai întâi apel la *FinanceRepository* pentru a obține informațiile financiare ale utilizatorului. Dacă informațiile sunt valide, serviciul creează un nou obiect *Account* cu datele contului și îl adaugă în baza de date prin intermediul *AccountRepository*. În final, serviciul returnează un mesaj de succes către controller, care îl trimite la frontend.

2. Ștergerea unui cont

Utilizatorul decide să șteargă un cont existent din frontend. Acesta declanșează cererea de ștergere care trimite o cerere HTTP DELETE către endpoint-ul */api/AccountController/{accountId}* din backend. Controllerul *AccountController* primește cererea de ștergere a contului și invocă metoda *DeleteAccountAsync* din serviciul *IAccountService*.

Serviciul *AccountService* implementează logica principală de ștergere a unui cont. Metoda verifică mai întâi dacă contul există în baza de date prin intermediul *AccountRepository*. Dacă contul este găsit, acesta este șters din baza de date, împreună cu toate cheltuielile și rapoartele asociate contului. În final, serviciul returnează un mesaj de succes către controller, care îl trimite la frontend.

3. Adăugarea unei cheltuieli

Cheltuielile pot fi adăugate sau șterse pentru fiecare cont. Fiecare cheltuială adăugată este afișată în lista de cheltuieli recente.

Utilizatorul completează formularul de adăugare a unei cheltuieli în frontend.

```
addExpense(accountId: number | null): void {  
  if (accountId === null) {  
    console.error('Account ID is null');  
    return;  
  }  
  const expenseDto = {  
    accountId: accountId,
```

```

amount: parseFloat(this.expenseForm.value.amount), // Ensure the amount is a number
date: this.expenseForm.value.date,
description: this.expenseForm.value.description
};
this.financeService.addExpense(expenseDto).subscribe(() => {
  this.loadFinance();
  this.expenseForm.reset({ date: new Date().toISOString().split('T')[0] });
  this.closeAddExpenseModal();
  window.location.reload(); //reload page
}, error => {
  console.error('Error adding expense:', error);
});
}

```

Funcția *addExpense* are rolul de a adăuga o cheltuială nouă pentru un cont specific. Pentru a realiza acest lucru, funcția primește ca parametru *accountId*, care reprezintă ID-ul contului la care se va adăuga cheltuiala. Datele necesare pentru crearea unei cheltuieli sunt preluate din formularul de cheltuieli și includ suma (*amount*), data (*date*) și descrierea (*description*). Înainte de a crea un obiect *expenseDto*, funcția verifică dacă *accountId* nu este null. Dacă este null, se afișează un mesaj de eroare în consolă și funcția se oprește. În caz contrar, se creează obiectul *expenseDto* cu valorile din formular și se trimite o cerere la serviciul *financeService* pentru a adăuga cheltuiala.

Datele introduse sunt trimise printr-o cerere HTTP POST către endpoint-ul */api/ExpenseController* din backend. Controllerul *ExpenseController* primește cererea de creare a cheltuielii și este responsabil pentru validarea inițială a datelor primite. Dacă datele sunt valide, controllerul invocă metoda *AddExpenseAsync* din serviciul *IExpenseService*. În cazul în care datele nu sunt valide, controllerul returnează un mesaj de eroare adecvat către frontend.

Serviciul *ExpenseService* implementează logica principală de creare a unei cheltuieli. Această metodă face apel la *AccountRepository* pentru a verifica existența contului și pentru a obține informațiile contului. Dacă verificările sunt trecute, serviciul creează un nou obiect *Expense* cu datele cheltuielii și îl adaugă în baza de date prin intermediul *ExpenseRepository*. După adăugarea cheltuielii, serviciul adaugă și un eveniment în calendarul utilizatorului prin intermediul *EventService*. În final, serviciul returnează un mesaj de succes către controller, care îl trimite la frontend.

La primirea răspunsului de succes, se încarcă din nou datele financiare pentru a reflecta noua cheltuială, se resetează formularul la valorile implicite și se închide modalul de adăugare a cheltuielii. În final, pagina se reîncarcă pentru a reflecta modificările efectuate.

4. Ștergerea unei cheltuieli

Utilizatorul decide să șteargă o cheltuială existentă din frontend și se activează funcția:

```
deleteExpense(accountId: number, expenseId: number): void {
  this.financeService.deleteExpense(expenseId).subscribe(() => {
    this.loadFinance();
    this.expenseReportsFromDb[accountId].forEach(report => {
      const reportExpenses = this.allExpenses[accountId].filter(expense => new
Date(expense.date).toISOString().substring(0, 7) === report.month);
      if (reportExpenses.length === 0) {
        this.reportService.deleteReport(report.id).subscribe(() => {
          console.log(`Report ${report.id} deleted successfully`);
          this.loadReports(accountId); // Reincarca rapoartele dupa stergere
          window.location.reload();
        }, error => {
          console.error('Error deleting report:', error);
        });
      }
    });
  });
}
```

Funcția *deleteExpense* este responsabilă pentru ștergerea unei cheltuieli existente dintr-un cont specific. Aceasta primește ca parametri *accountId* și *expenseId*, reprezentând ID-ul contului și respectiv ID-ul cheltuielii care urmează să fie ștearsă. Funcția trimite o cerere la serviciul *financeService* pentru a șterge cheltuiala.

Acesta declanșează cererea de ștergere care trimite o cerere HTTP DELETE către endpoint-ul */api/ExpenseController/{expenseId}* din backend. Controllerul *ExpenseController* primește cererea de ștergere a cheltuielii și invocă metoda *DeleteExpenseAsync* din serviciul *IExpenseService*.

Serviciul *ExpenseService* implementează logica principală de ștergere a unei cheltuieli. Metoda verifică mai întâi dacă cheltuiala există în baza de date prin intermediul *ExpenseRepository*. Dacă cheltuiala este găsită, aceasta este ștearsă din baza de date, iar serviciul șterge și evenimentul asociat din calendar prin intermediul *EventService*. În final, serviciul returnează un mesaj de succes către controller, care îl trimite la frontend.

La primirea răspunsului de succes, funcția încarcă din nou datele financiare pentru a reflecta ștergerea cheltuielii. De asemenea, se verifică rapoartele de cheltuieli pentru a șterge rapoartele care nu mai conțin cheltuieli. În cazul în care toate cheltuielile dintr-un raport au fost șterse, funcția șterge raportul gol (face un alt apel către backend) pentru a menține datele actualizate și corecte.

5. Adăugarea de fonduri

Utilizatorul adaugă fonduri într-un cont existent prin intermediul formularului din frontend. Datele introduse sunt trimise printr-o cerere HTTP POST către endpoint-ul */api/AccountController/addFunds* din backend. Controllerul *AccountController* primește cererea de adăugare a fondurilor și este responsabil pentru validarea inițială a datelor primite. Dacă datele sunt valide, controllerul invocă metoda *AddFundsAsync* din serviciul *IAccountService*.

Serviciul *AccountService* implementează logica principală de adăugare a fondurilor.

```
public async Task<bool> AddFundsAsync(int accountId, decimal amount)
{
    var account = await _accountRepository.GetAccountByIdAsync(accountId);
    if (account == null || account.Finance == null || account.Finance.User ==
null || account.Finance.User.Calendar == null)
        return false;

    account.Amount += amount;
    await _accountRepository.UpdateAsync(account);

    // Creăm un nou Event pentru adăugarea fondurilor
    var newEvent = new Event
    {
        CalendarId = account.Finance.User.Calendar.Id, // Asigură-te că
entitățile sunt încărcate corect
        Date = DateTime.Now,
        Description = "Added funds",
        Type = "deposit",
        AccountId = accountId,
        Amount = amount
    };

    await _eventService.AddEventAsync(newEvent);

    // Verifică dacă sunt obiective atinse
    await CheckGoalsAsync(account);

    return true;
}
```

Această metodă face apel la *AccountRepository* pentru a obține informațiile contului și pentru a actualiza balanța contului.

```
public async Task UpdateAsync(Account account)
{
    _context.Accounts.Update(account);
    await _context.SaveChangesAsync();
}
```

După actualizarea balanței, serviciul creează un nou eveniment de depozit în calendarul utilizatorului prin intermediul *EventService*. După, metoda *AddFundsAsync* verifică dacă există obiective de economii (goals) asociate utilizatorului. Această verificare este realizată prin metoda *CheckGoalsAsync*, care este apelată automat după actualizarea balanței contului:

Metoda *CheckGoalsAsync* face apel la *GoalRepository* pentru a obține toate obiectivele asociate utilizatorului. Pentru fiecare obiectiv, se verifică dacă suma din contul de economii atinge sau depășește suma obiectivului. Dacă un obiectiv este atins, statusul acestuia este actualizat la "completed" și se creează un nou obiect *Achievement* pentru utilizator, care este adăugat în baza de date prin intermediul *AchievementService*. În cazul în care un obiectiv nu este atins sau data limită a obiectivului a trecut, obiectivul este șters din baza de date.

În final, serviciul returnează un mesaj de succes către controller, care îl trimite la frontend.

Interacțiuni frontend backend nemijlocite de user

1. Generarea rapoartelor lunare

```
generateMonthlyReports(accountId: number): void {
    const expenses = this.allExpenses[accountId];
    const monthlyExpenses: { [key: string]: any[] } = {};

    expenses.forEach(expense => {
        const month = new Date(expense.date).toISOString().substring(0, 7); // Get month in format 'YYYY-MM'
        if (!monthlyExpenses[month]) {
            monthlyExpenses[month] = [];
        }
        monthlyExpenses[month].push(expense);
    });

    Object.keys(monthlyExpenses).forEach(month => {
        const filteredExpenses = monthlyExpenses[month];
        const totalExpenses = filteredExpenses.reduce((sum: number, expense: any) => sum + expense.amount, 0);
        const numberOfExpenses = filteredExpenses.length;

        const report = {
            accountId: accountId,
```

```

    month: month + "-01T00:00:00Z", // Ensure the month is in 'YYYY-MM-DDTHH:mm:ssZ' format
    totalExpenses: totalExpenses,
    numberOfExpenses: numberOfExpenses
  };

  this.reportService.createOrUpdateReport(report).subscribe(() => {
    console.log(`Monthly report for ${month} created or updated successfully for account ${accountId}`);
  }, error => {
    console.error('Error creating or updating monthly report: ', error);
  });
});
}

```

Funcția *generateMonthlyReports* este utilizată pentru a genera rapoarte lunare de cheltuieli pentru un cont specific. Aceasta preia toate cheltuielile unui cont, le sortează în funcție de lună și calculează totalul cheltuielilor și numărul de cheltuieli pentru fiecare lună. Funcția parcurge fiecare cheltuială și obține luna în format YYYY-MM. Apoi, adaugă cheltuielile în obiectul *monthlyExpenses*, unde cheile reprezintă lunile. După ce toate cheltuielile sunt sortate pe luni, funcția calculează totalul cheltuielilor și numărul de cheltuieli pentru fiecare lună. Aceste informații sunt utilizate pentru a crea un raport lunar, care este trimis la serviciul *reportService* pentru a fi creat sau actualizat în baza de date. Funcția *generateMonthlyReports* se asigură că rapoartele lunare sunt create sau actualizate cu succes și afișează un mesaj în consolă pentru fiecare raport creat sau actualizat.

Pagina încarcă în mod automat rapoartele financiare pe fiecare lună pentru fiecare cont al utilizatorului, dacă există un astfel de raport, oferind utilizatorilor o vedere detaliată asupra cheltuielilor din fiecare lună.

O cerere HTTP GET este trimisă către endpoint-ul */api/ReportController/account/{accountId}* din backend. Controllerul *ReportController* primește cererea de obținere a rapoartelor lunare și invocă metoda *GetReportsByAccountIdAsync* din serviciul *IReportService*.

Serviciul *ReportService* implementează logica principală de obținere a rapoartelor lunare. Metoda face apel la *ReportRepository* pentru a obține toate rapoartele lunare asociate contului specificat. Rapoartele sunt returnate către controller, care le trimite în răspunsul HTTP către frontend pentru a fi afișate utilizatorului.

2. Încărcarea cheltuielilor pentru fiecare cont

Încărcarea conturilor și a cheltuielilor pentru fiecare cont se realizează automat atunci când utilizatorul accesează pagina de finanțe. Aceste date sunt folosite pentru a genera rapoartele de cheltuieli.

Funcția `loadFinance` din frontend trimite o cerere HTTP GET către endpoint-ul `/api/FinanceController/{userId}` din backend pentru a obține toate conturile utilizatorului și detaliile acestora. Controllerul `FinanceController` primește cererea și invocă metoda `GetFinanceByUserIdAsync` din serviciul `IFinanceService`.

Serviciul `FinanceService` gestionează logica principală de obținere a detaliilor financiare ale utilizatorului. Metoda face apel la `FinanceRepository` pentru a obține toate conturile asociate utilizatorului, împreună cu informațiile necesare despre fiecare cont.

După ce datele conturilor sunt obținute, frontend-ul apelează metodele `loadExpenses` și `loadReports` pentru fiecare cont în parte. Funcția `loadExpenses` trimite o cerere HTTP GET către endpoint-ul `/api/AccountController/{accountId}/expenses` pentru a obține toate cheltuielile asociate contului respectiv. Controllerul `AccountController` primește cererea și invocă metoda `GetAllExpensesByAccountIdAsync` din serviciul `IExpenseService`.

Serviciul `ExpenseService` gestionează logica principală de obținere a cheltuielilor asociate unui cont. Metoda face apel la `ExpenseRepository` pentru a obține toate cheltuielile din baza de date pentru contul specificat. Cheltuielile sunt returnate către controller, care le trimite în răspunsul HTTP către frontend pentru a fi afișate utilizatorului.

3. Încărcarea ultimelor 3 cheltuieli

În mod similar, are loc încărcarea ultimelor 3 cheltuieli pentru a fi afișate pe cardul conturilor. Aceasta se face pentru a oferi utilizatorului o vizualizare rapidă a cheltuielilor recente.

La inițializarea componentei, frontend-ul trimite o cerere HTTP GET către endpoint-ul `/api/AccountController/{accountId}/lastThreeExpenses` pentru a obține ultimele 3 cheltuieli pentru fiecare cont. Controllerul `AccountController` primește cererea și invocă metoda `GetLastThreeExpensesByAccountIdAsync` din serviciul `IExpenseService`.

Serviciul *ExpenseService* gestionează logica principală de obținere a cheltuielilor recente asociate unui cont. Metoda face apel la *ExpenseRepository* pentru a obține ultimele 3 cheltuieli din baza de date pentru contul specificat. Cheltuielile sunt returnate către controller, care le trimite în răspunsul HTTP către frontend pentru a fi afișate utilizatorului.

3.4.4 Pagina Goals.

Pagina de obiective (Figura 3 12) permite utilizatorilor să își stabilească și să își monitorizeze obiectivele financiare pe termen lung și scurt.

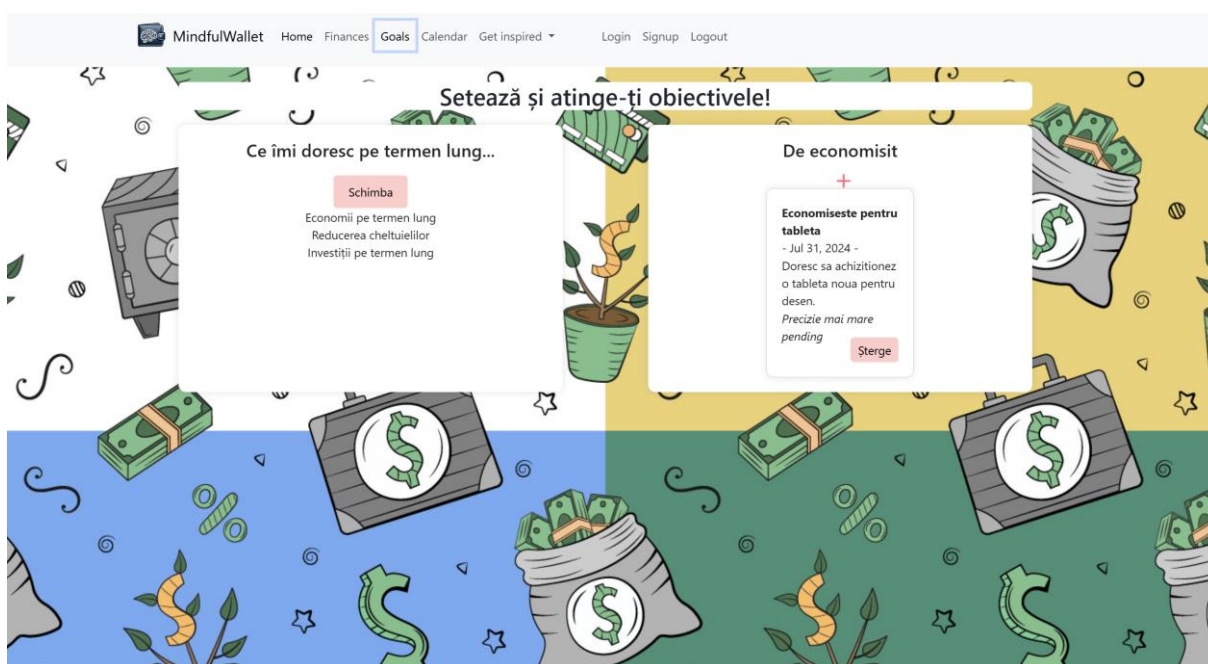


Figura 3 12

Pagina de obiective este structurată în mai multe secțiuni, fiecare cu rolul său specific:

Un container principal conține toate elementele esențiale ale paginii, inclusiv titlul și secțiunile de obiective.

Pagina descrie două secțiuni de obiective: obiective pe termen lung și obiective pe termen scurt.

Obiectivele pe termen lung: utilizatorii pot deschide un modal pentru a modifica obiectivele pe termen lung folosind butonul "Schimba". Aceste tipuri de obiective sunt stocate local, nefiind trimise în baza de date.

Serviciul *GoalService* implementează logica de stocare a obiectivelor pe termen lung folosind *localStorage* sau o altă metodă de persistență pe frontend. Dacă salvarea este reușită, modalul este închis și lista de obiective este actualizată.

Secțiunea de obiective specifice afișează obiectivele de economisire definite de user. Fiecare obiectiv este afișat într-un card care conține detalii despre obiectiv, inclusiv titlul, descrierea, motivația, data scadentă și statusul. Utilizatorii pot adăuga noi obiective sau șterge obiective existente.

Si această pagină utilizează modale: unul pentru obiective pe termen lung care permite utilizatorilor să selecteze sau să deselecteze obiective pe termen lung dintr-o listă de opțiuni, iar modalul pentru obiective specifice permite utilizatorilor să adauge un nou obiectiv specific completând un formular cu titlul, descrierea, motivația, data și suma necesară.

Interacțiuni User-Frontend-Backend

Componenta *GoalsComponent* este responsabilă pentru gestionarea logicii paginii de obiective. Aceasta utilizează mai multe servicii pentru a obține și manipula datele: *GoalService* gestionează cererile legate de obiective, cum ar fi obținerea, adăugarea și ștergerea obiectivelor și *AuthService* care se ocupa cu autentificarea utilizatorului și furnizează informații despre utilizator, cum ar fi identificatorul (*userId*).

1. Adăugarea unui obiectiv nou:

Utilizatorul accesează pagina de obiective și dorește să adauge un obiectiv nou. Acesta completează formularul de pe frontend pentru adăugarea unui obiectiv specific.

```
saveSpecificGoal() {
  this.newGoal.userId = this.userId;
  this.goalService.addGoal(this.newGoal).subscribe(
    response => {
      this.specificGoals.push(response);
      this.newGoal = { id: 0, userId: 0, title: '', description: '',
motivation: '', dueDate: new Date(), amount: 0, status: 'pending' };
      this.closeSpecificGoalModal();
    },
    error => {
      console.error('Error adding goal:', error);
    }
  );
}
```

În metoda dată, formularul din frontend folosește Angular Two-Way Data Binding pentru a prelua și a trimite datele obiectivului specific către backend. Formularul utilizează directivele [(ngModel)] pentru a lega datele din câmpurile formularului la proprietățile obiectului newGoal.

Datele introduse de utilizator sunt trimise printr-o cerere HTTP POST către endpoint-ul /api/GoalController din backend. Controllerul GoalController primește cererea de creare a unui obiectiv nou și este responsabil pentru validarea inițială a datelor primite. Dacă datele sunt valide, controllerul invocă metoda AddGoalAsync din serviciul IGoalService. În cazul în care datele nu sunt valide, controllerul returnează un mesaj de eroare adecvat către frontend.

Serviciul GoalService implementează logica principală de creare a unui obiectiv. Această metodă face apel la GoalRepository pentru a adăuga noul obiectiv în baza de date. În final, serviciul returnează un mesaj de succes către controller, care îl trimite la frontend.

2. Ștergerea unui obiectiv:

Utilizatorul poate decide să șteargă un obiectiv existent din frontend.

```
deleteGoal(goalId: number) {  
  this.goalService.deleteGoal(goalId).subscribe(  
    () => {  
      this.specificGoals = this.specificGoals.filter(goal => goal.id !==  
goalId);  
    },  
    error => {  
      console.error('Error deleting goal:', error);  
    }  
  );  
}
```

Acesta declanșează cererea de ștergere care trimite o cerere HTTP DELETE către endpoint-ul /api/GoalController/{goalId} din backend. Controllerul GoalController primește cererea de ștergere a obiectivului și invocă metoda DeleteGoalAsync din serviciul IGoalService.

Serviciul GoalService implementează logica principală de ștergere a unui obiectiv. Metoda verifică mai întâi dacă obiectivul există în baza de date prin intermediul GoalRepository.

Dacă obiectivul este găsit, acesta este șters din baza de date. În final, serviciul returnează un mesaj de succes către controller, care îl trimite la frontend.

Interacțiuni frontend backend nemijlocite de user

1. Incarcarea Goal-urilor

Încărcarea obiectivelor pentru utilizator se realizează automat atunci când utilizatorul accesează pagina de obiective. Funcția `loadSpecificGoals` din frontend trimite o cerere HTTP GET către endpoint-ul `/api/GoalController/{userId}` din backend pentru a obține toate obiectivele utilizatorului. Controllerul `GoalController` primește cererea și invocă metoda `GetGoalsByUserIdAsync` din serviciul `IGoalService`.

Serviciul `GoalService` gestionează logica principală de obținere a obiectivelor utilizatorului. Metoda face apel la `GoalRepository` pentru a obține toate obiectivele din baza de date pentru utilizatorul specificat. Obiectivele sunt returnate către controller, care le trimite în răspunsul HTTP către frontend pentru a fi afișate utilizatorului.

2. Verificarea atingerii obiectivelor

Deși acest lucru nu ține de `GoalService`, voi menționa și faptul că verificarea atingerii unui obiectiv se face automat în backend atunci când se adaugă fonduri într-un cont de economii. Am discutat despre acest lucru în subcapitolul 3.4.3.

3.4.5 Pagina Calendar.

Calendarul din aplicația "Mindful Wallet" (figura 3 13) este conceput pentru a ajuta utilizatorii să vizualizeze obiectivele financiare și evenimentele importante într-un mod organizat și ușor de utilizat. Acesta afișează atât obiectivele pe termen lung, cât și evenimentele financiare specifice, pe baza datelor obținute din backend.

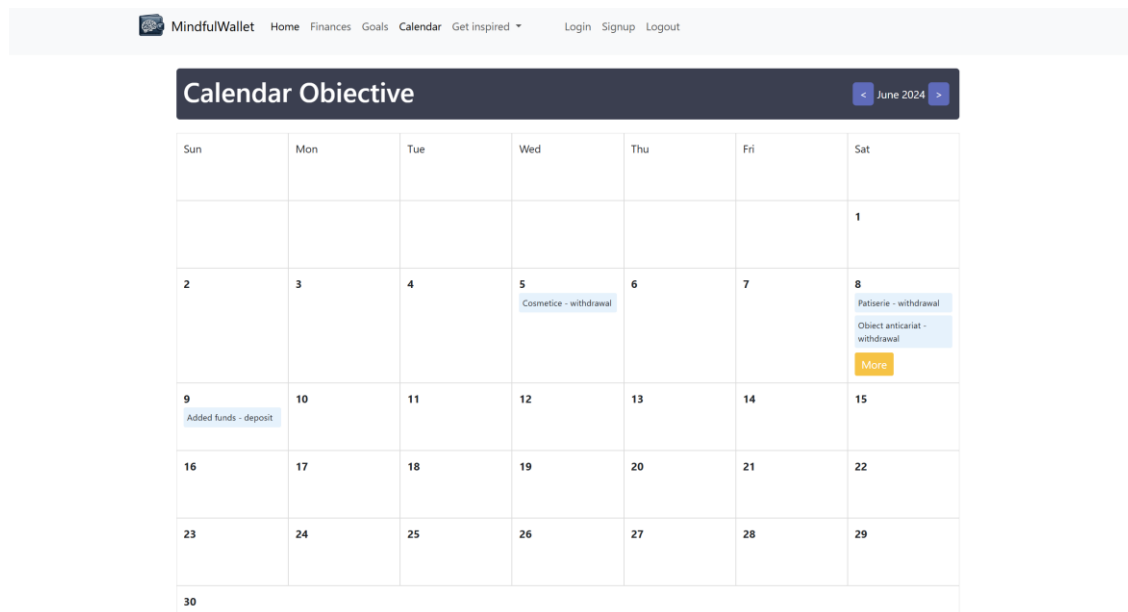


Figura 3 13

Componenta de calendar (*CalendarComponent*) este responsabilă pentru afișarea calendarului și a datelor aferente acestuia. Aceasta include un antet cu controale pentru navigarea între lunile calendarului și o structură grilă pentru afișarea zilelor săptămânii și a evenimentelor.

Calendarul este generat dinamic pe baza lunii și anului curent, iar utilizatorul poate naviga între luni folosind butoanele "Previous" și "Next". Fiecare zi din calendar poate afișa data, evenimentele și obiectivele asociate, iar dacă există mai multe evenimente într-o zi, utilizatorul poate deschide un modal pentru a vizualiza detaliile acestora.

Interacțiuni User-Frontend-Backend

Când utilizatorul accesează pagina de calendar, funcția *ngOnInit()* din componenta *CalendarComponent* este declanșată, care obține identificadorul utilizatorului prin intermediul *AuthService*. Odată ce *userId* este obținut, sunt apelate metodele *loadEvents()* și *loadSpecificGoals()* pentru a încărca evenimentele și obiectivele utilizatorului.

Funcția *loadEvents()* trimite o cerere HTTP GET către endpoint-ul */api/EventController/{userId}* din backend pentru a obține toate evenimentele asociate utilizatorului. Controllerul *EventController* primește cererea și invocă metoda *GetEventsByUserIdAsync* din serviciul *IEventService*. Serviciul *EventService* gestionează logica principală de obținere a evenimentelor utilizatorului. Metoda face apel la

EventRepository pentru a obține toate evenimentele din baza de date pentru utilizatorul specificat și le returnează către frontend pentru a fi afișate în calendar.

Funcția *loadSpecificGoals()* trimite o cerere HTTP GET către endpoint-ul */api/GoalController/{userId}* din backend pentru a obține toate obiectivele specifice ale utilizatorului. Controllerul *GoalController* primește cererea și invocă metoda *GetGoalsByUserIdAsync* din serviciul *IGoalService*. Serviciul *GoalService* gestionează logica principală de obținere a obiectivelor specifice ale utilizatorului. Metoda face apel la *GoalRepository* pentru a obține toate obiectivele din baza de date pentru utilizatorul specificat și le returnează către frontend pentru a fi afișate în calendar.

Generarea calendarului:

Funcția *generateCalendar()* este responsabilă pentru generarea calendarului pe baza anului și lunii curente. Aceasta calculează prima zi a lunii, ultima dată a lunii și grupează zilele în săptămâni. Pentru fiecare zi, funcția filtrează evenimentele și obiectivele specifice pentru a le afișa corespunzător în calendar.

3.4.6 Pagina Home.

Pagina de home a aplicației "Mindful Wallet" (Figura 3 14) este concepută pentru a oferi utilizatorilor o vizualizare centralizată a informațiilor financiare relevante, cum ar fi sfaturi, obiective, rapoarte și realizări. Această pagină este construită utilizând Angular pentru frontend și diverse servicii backend pentru a asigura o experiență de utilizator fluidă și eficientă.

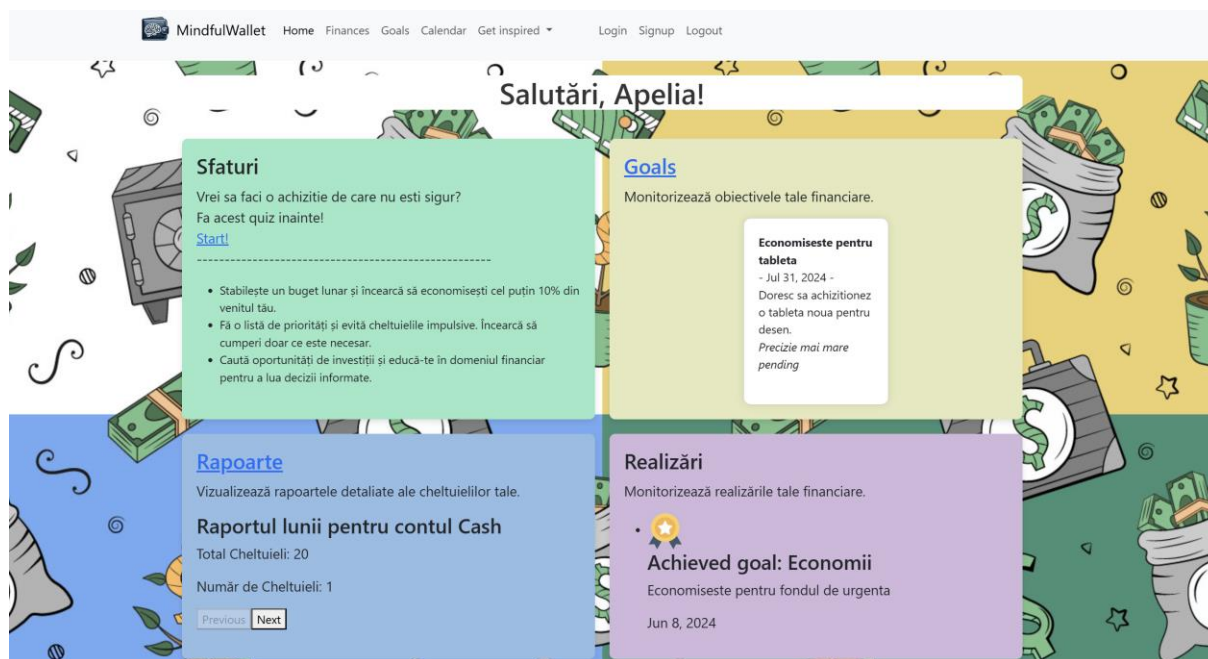


Figura 3 14

În interiorul containerului principal, un header afișează un mesaj de bun venit personalizat pentru utilizator, folosind variabila *userName*. Structura paginii include mai multe secțiuni distincte: sfaturi, obiective, rapoarte și realizări, fiecare având un scop specific.

Secțiunea "Sfaturi" afișează sfaturi financiare personalizate pentru utilizator, ajutându-l să ia decizii informate. Aceste sfaturi sunt listate din variabila *personalizedAdvice*.

Secțiunea "Goals" permite utilizatorului să monitorizeze și să gestioneze obiectivele financiare. Obiectivele sunt afișate într-un format de card, fiecare card conținând detalii precum titlul, data scadentă, descrierea, motivația și statusul obiectivului.

Secțiunea "Rapoarte" afișează rapoarte detaliate ale cheltuielilor pentru conturile utilizatorului, incluzând totalul cheltuielilor și numărul de cheltuieli pentru luna curentă.

Nu în ultimul rând, secțiunea "Realizări" prezintă realizările financiare ale utilizatorului, inclusiv imagini și descrieri ale realizărilor obținute.

Stilizarea fiecărei secțiuni este distinctă pentru a le diferenția vizual: sfaturile au un fundal verde deschis, obiectivele un fundal galben deschis, rapoartele un fundal albastru deschis și realizările un fundal mov deschis. Aceste stiluri sunt definite în fișierul CSS asociat.

Interacțiuni Frontend-Backend nemijlocite de utilizator

Componenta *HomeComponent* face apeluri la *AuthService* și *HomeService* pentru a aduce în frontend informațiile relevante de afișat:

1. Încărcarea automată a conturilor și rapoartelor

La inițializarea componentei *HomeComponent*, metoda *ngOnInit()* apelează serviciul *AuthService* pentru a obține *userId*. Cu *userId*, frontend-ul trimite o cerere HTTP GET către endpoint-ul `/api/Account/user/{userId}` din backend pentru a obține conturile utilizatorului.

Endpoint-ul `/api/Account/user/{userId}` este gestionat de *AccountController*, care primește cererea și invocă metoda *GetAccountsByUserId* din serviciul *IAccountService*. Serviciul *AccountService* face apel la *AccountRepository* pentru a obține toate conturile asociate utilizatorului din baza de date și returnează rezultatele către *AccountController*, care le trimite înapoi la frontend. Odată ce conturile sunt primite, frontend-ul trimite cereri HTTP GET către endpoint-ul `/api/Report/current/{accountId}` pentru fiecare cont pentru a obține rapoartele lunare curente.

Endpoint-ul `/api/Report/current/{accountId}` este gestionat de *ReportController*, care primește cererea și invocă metoda *GetCurrentMonthReportAsync* din serviciul *IReportService*. Serviciul *ReportService* face apel la *ReportRepository* pentru a obține raportul curent pentru contul specificat și returnează rezultatele către *ReportController*, care le trimite înapoi la frontend.

2. Generarea sfaturilor personalizate

Sfaturile personalizate sunt generate automat pe baza obiectivelor pe termen lung ale utilizatorului. Această funcționalitate utilizează datele stocate în *localStorage* pentru a genera sfaturi relevante și utile, fără a necesita intervenția directă a utilizatorului.

Se obțin obiectivele pe termen lung din *localStorage* folosind *GoalService*. *GoalService* furnizează un stream de date (*longTermGoals\$*) prin intermediul unui *BehaviorSubject*, care conține obiectivele pe termen lung stocate în *localStorage*. În funcție de obiectivele pe termen lung, se generează sfaturi personalizate utilizând o listă predefinită de sfaturi pentru fiecare obiectiv.

3. Afișarea realizărilor și obiectivelor specifice

Realizările și obiectivele specifice ale utilizatorului sunt încărcate automat din baza de date și afișate în pagina de Home. Această funcționalitate oferă utilizatorilor o vizualizare clară a progresului lor financiar și îi motivează să continue să atingă noi obiective.

Frontend-ul trimite o cerere HTTP GET către endpoint-ul `/api/Account/user/{userId}/achievements` pentru a obține realizările utilizatorului. De asemenea, trimite o cerere HTTP GET către endpoint-ul `/api/Goal/{userId}` pentru a obține obiectivele specifice ale utilizatorului.

Endpoint-ul `/api/Account/user/{userId}/achievements` este gestionat de `AccountController`, care primește cererea și invocă metoda `GetAchievementsByUserId` din serviciul `IAchievementService`. Endpoint-ul `/api/Goal/{userId}` este gestionat de `GoalController`, care primește cererea și invocă metoda `GetGoalsByUserId` din serviciul `IGoalService`. Serviciul `AchievementService` face apel la `AchievementRepository` pentru a obține toate realizările asociate utilizatorului și le returnează către `AccountController`, care le trimite înapoi la frontend. Serviciul `GoalService` face apel la `GoalRepository` pentru a obține toate obiectivele specifice ale utilizatorului și le returnează către `GoalController`, care le trimite înapoi la frontend.

3.4.7 Alte pagini

Pagina de Start

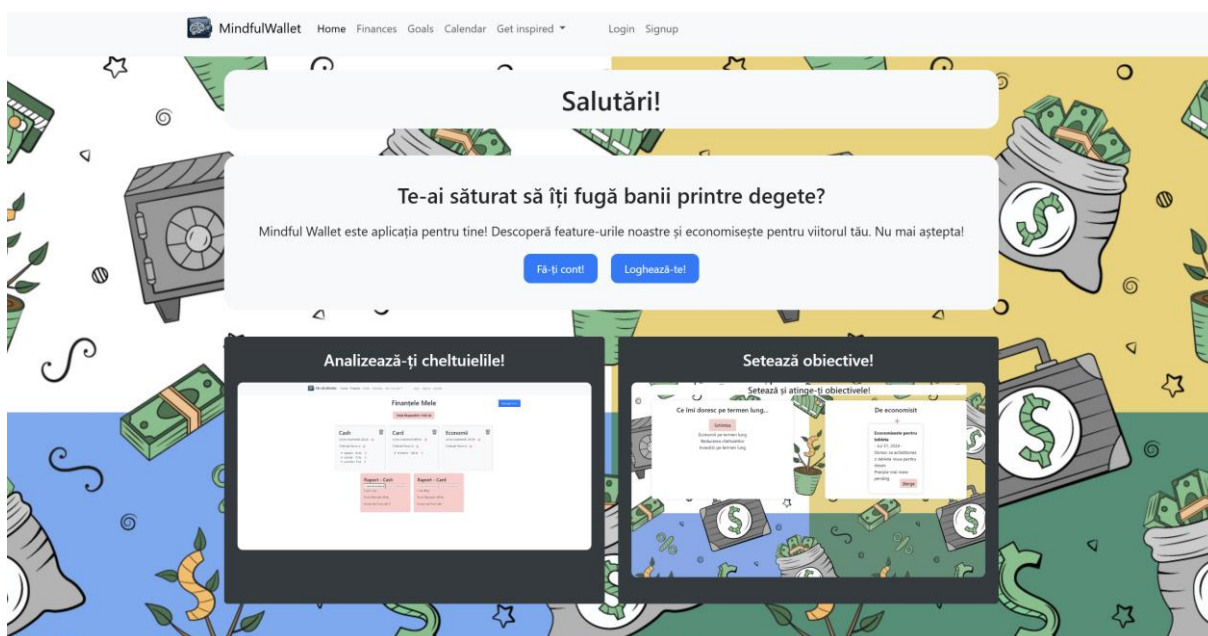


Figura 3 15

Pagina de start a aplicației Mindful Wallet (Figura 3 15) este concepută pentru a introduce utilizatorii în funcționalitățile aplicației și pentru a-i încuraja să își creeze un cont sau să se logheze. Structura acestei pagini este simplă și atractivă, oferind un mesaj de bun venit și evidențiind principalele caracteristici ale aplicației.

Antetul salută utilizatorii cu un mesaj de bun venit. Secțiunea introductivă le prezintă utilizatorilor avantajele utilizării Mindful Wallet, cum ar fi posibilitatea de a economisi bani și de a-și gestiona mai eficient finanțele personale. Aceasta este urmată de două butoane call-to-action: unul pentru a crea un cont și altul pentru a se loga, direcționând utilizatorii către paginile de înregistrare și autentificare.

În continuare, sunt prezentate două funcționalități principale ale aplicației: analizarea cheltuielilor și setarea obiectivelor financiare. Fiecare funcționalitate este ilustrată cu o imagine reprezentativă pentru a capta atenția utilizatorilor.

Pagina de Quiz

The screenshot shows a web application interface for a quiz. At the top, there is a navigation bar with the logo 'MindfulWallet' and links for 'Home', 'Finances', 'Goals', 'Calendar', 'Get inspired', 'Login', 'Signup', and 'Logout'. The main content area is titled 'Afla daca ar trebui să cumperi' (Find out if you should buy). It contains a series of questions, each with 'Da' (Yes) and 'Nu' (No) radio button options. The questions are: 1. 'Iti doresti produsul?' (Do you want the product?). 2. 'Această achiziție va face viața mai bună sau mai ușoară?' (This purchase will make life better or easier?). 3. 'Chiar AI NEVOIE de acest articol?' (Do you really NEED this item?). 4. 'Ți-ai permis să cumperi produsul?' (Did you allow yourself to buy the product?). 5. 'Ți-a aduce bucurie?' (Will it bring you joy?). 6. 'Ai mai folosit un produs similar înainte?' (Have you used a similar product before?). 7. 'Este un produs de calitate?' (Is it a quality product?). 8. 'Ai verificat recenzile produsului?' (Have you checked the product reviews?). 9. 'Achiziționarea acestui produs se aliniază cu obiectivele tale financiare pe termen lung?' (Buying this product aligns with your long-term financial goals?). The background of the page is decorated with various financial-themed illustrations like a safe, money bags, coins, and a briefcase.

Figura 3 16

Pagina de quiz (Figura 3 16) este concepută pentru a ajuta utilizatorii să decidă dacă ar trebui să facă o anumită achiziție. Aceasta conține o serie de întrebări menite să le evalueze nevoile și dorințele legate de achiziția respectivă.

Utilizatorii sunt invitați să răspundă la fiecare întrebare cu "da" sau "nu". Răspunsurile sunt colectate și procesate pentru a genera un scor total, care determină mesajul final de feedback. Acest mesaj poate sugera dacă achiziția este utilă, dacă utilizatorii ar trebui să se mai gândească, sau dacă nu ar trebui să facă achiziția respectivă.

Pagina de quiz include și un modal pentru afișarea rezultatului, oferind utilizatorilor un feedback clar și concis bazat pe răspunsurile lor.

3.5 Testarea și validarea

În dezvoltarea aplicației Mindful Wallet, am realizat personal testarea manuală pentru a asigura calitatea și funcționalitatea corectă. Testarea manuală a implicat verificarea fiecărei funcționalități a aplicației prin interacțiuni directe și observații, pentru a mă asigura că toate componentele funcționează conform specificațiilor.

Metodologia de Testare Manuală

Procesul de testare manuală a urmat mai multe etape esențiale. Am început cu planificarea testelor, definind obiectivele și identificând scenariile critice. Am stabilit un set de cazuri de testare pentru fiecare funcționalitate a aplicației, acoperind aspecte precum autentificarea, gestionarea conturilor financiare, setarea și monitorizarea obiectivelor și interfața de utilizare.

Executarea testelor a implicat navigarea prin fiecare pagină a aplicației pentru a verifica corectitudinea elementelor UI și a funcționalităților. Am introdus date valide și invalide în formulare pentru a testa validările și fluxurile de utilizare, verificând interacțiunea corectă între frontend și backend prin testarea API-urilor utilizate de aplicație.

În timpul testării, am documentat erorile și problemele întâlnite, clasificându-le pe categorii (critice, majore, minore) pentru a prioritiza corecțiile. Am colaborat cu echipa de dezvoltare pentru rezolvarea problemelor identificate și am verificat corecțiile implementate pentru a mă asigura că problemele au fost rezolvate.

Scenarii de Testare Acoperite

Testarea manuală a acoperit diverse scenarii. Pentru autentificare și autorizare, am verificat procesul de creare a unui cont nou, autentificarea cu date corecte și incorecte, resetarea parolei și utilizarea token-ului de resetare. Gestionarea conturilor financiare a inclus adăugarea, modificarea și ștergerea conturilor, verificarea balanței după adăugarea și eliminarea cheltuielilor și generarea rapoartelor de cheltuieli.

Am testat setarea și monitorizarea obiectivelor financiare pe termen lung și scurt, afișarea acestora în calendarul utilizatorului și actualizarea stării pe baza progresului realizat. Interfața de utilizare a fost verificată prin navigarea între paginile aplicației, verificarea afișării corecte a informațiilor și elementelor grafice și testarea compatibilității cu diferite rezoluții și dispozitive etc.

Concluzii

4.1 Rezultate obtinute

Proiectul Mindful Wallet este mai mult decât o simplă aplicație financiară; este o demonstrație a puterii unei arhitecturi bine gândite și a unui design centrat pe utilizator. Implementarea arhitecturii Clean și Component-Based a fost esențială pentru a crea o aplicație robustă, flexibilă și ușor de întreținut.

Arhitectura Clean aplicată în backend a permis o structurare clară și logică a aplicației. Fiecare strat al aplicației, de la expunerea API-ului web și gestionarea cererilor HTTP, până la logica de business și accesul la date, a avut un rol bine definit. Această separare a responsabilităților a făcut posibilă dezvoltarea și testarea aplicației într-un mod eficient, permițând modificări rapide și adăugarea de noi funcționalități fără a afecta alte părți ale sistemului.

În frontend, arhitectura bazată pe componente a asigurat o organizare clară și eficientă a codului. Fiecare componentă, de la autentificare și înregistrare, până la gestionarea cheltuielilor și vizualizarea rapoartelor financiare, a fost concepută pentru a oferi utilizatorilor o experiență de interacțiune fluentă și intuitivă. Utilizarea validării reactive a formularelor și protecția rutelor prin gărzi de autentificare au contribuit la crearea unei interfețe sigure și prietenoase.

Designul interfeței cu utilizatorul a fost axat pe simplitate și funcționalitate. Paginile de login și signup, împreună cu modalul pentru resetarea parolei, au fost concepute pentru a oferi o experiență de utilizare plăcută și sigură.

Securitatea datelor utilizatorilor a fost un aspect esențial în dezvoltarea aplicației Mindful Wallet. Implementarea autentificării JWT, utilizarea token-urilor de reîmprospătare și hashuirea parolelor au asigurat protecția informațiilor sensibile și integritatea sistemului de autentificare. Aceste măsuri de securitate au prevenit accesul neautorizat și au protejat datele utilizatorilor împotriva atacurilor.

Structura modulară a aplicației a permis adăugarea ușoară de noi funcționalități și modificarea celor existente fără a afecta alte părți ale aplicației. Această flexibilitate asigură că Mindful Wallet este pregătit să facă față cerințelor viitoare și să se adapteze rapid la schimbările tehnologice.

Mindful Wallet demonstrează cum principiile arhitecturale moderne și un design centrat pe utilizator se pot asocia pentru a crea o aplicație financiară robustă, scalabilă și securizată.

4.2 Limitari si provocari

Mindful Wallet nu este lipsit de limitări. O limitare semnificativă constă în incapacitatea de a genera rapoarte mai detaliate pe baza categoriilor de produse sau servicii din care fac parte cheltuielile. În prezent, rapoartele financiare oferă o vedere generală asupra cheltuielilor, însă nu oferă detalii suficiente care să permită utilizatorilor să analizeze în profunzime modul în care își gestionează finanțele pe categorii specifice. Această funcționalitate ar putea îmbunătăți considerabil experiența utilizatorilor și ar putea oferi o perspectivă mai clară asupra obiceiurilor de consum.

Un alt aspect important îl reprezintă provocările întâmpinate în dezvoltarea aplicației. O provocare majoră a fost implementarea unor sfaturi personalizate pentru utilizatori, bazate pe achizițiile efectuate. O astfel de funcționalitate ar necesita analiza și interpretarea datelor de consum ale utilizatorilor, oferind recomandări care să îi ajute să-și optimizeze cheltuielile și să-și atingă obiectivele financiare. Implementarea acestui tip de personalizare nu este doar tehnic provocatoare, ci și necesită o atenție sporită la confidențialitatea și securitatea datelor utilizatorilor.

4.3 Sugestii pentru lucrari viitoare

În vederea continuării dezvoltării aplicației Mindful Wallet, se pot lua în considerare mai multe direcții de extindere și îmbunătățire. O primă sugestie este generarea de rapoarte financiare mai detaliate, care să includă analiza cheltuielilor pe categorii de produse și servicii. Aceasta ar permite utilizatorilor să obțină o imagine mai clară asupra modului în care își gestionează finanțele, identificând zonele unde cheltuiesc cel mai mult și unde ar putea economisi. De asemenea, se poate adăuga funcționalitatea de monitorizare a evoluției

economiilor utilizatorilor, oferindu-le informații utile despre progresul lor financiar și încurajându-i să-și mențină obiceiurile de economisire.

O îmbunătățire semnificativă a feature-urilor pentru Mindful Wallet ar fi condiționată de integrarea cu aplicațiile bancare, permițând astfel încărcarea automată a soldurilor conturilor. Această integrare ar simplifica gestionarea finanțelor pentru utilizatori, oferindu-le o actualizare automată și precisă a soldurilor lor bancare direct în aplicație. Utilizatorii nu ar mai trebui să introducă manual informațiile financiare, reducând astfel timpul și efortul necesar pentru gestionarea bugetului personal. Integrarea ar putea fi realizată prin utilizarea API-urilor bancare disponibile, asigurând astfel o sincronizare continuă și securizată a datelor financiare.

O altă direcție interesantă ar fi dezvoltarea unei extensii web pentru browsere, care să blocheze accesul utilizatorilor la site-urile de cumpărături frecvent accesate. Aceasta ar putea ajuta utilizatorii să reducă cheltuielile impulsive și să se concentreze mai mult pe economisire și gestionarea atentă a bugetului. Extensia ar putea oferi notificări și alternative constructive, cum ar fi sugestii pentru economii sau activități care nu implică cheltuieli.

În plus, implementarea unei pagini de magazin în cadrul aplicației Mindful Wallet ar putea adăuga valoare suplimentară pentru utilizatori. Aceasta pagină ar putea oferi produse utile pentru gestionarea finanțelor personale, cum ar fi agende de cheltuieli, portofele RFID și alte instrumente de organizare financiară. Prin oferirea acestor produse direct în aplicație, utilizatorii ar putea accesa ușor resursele necesare pentru a-și îmbunătăți gestionarea finanțelor.

Aceste sugestii ar putea contribui semnificativ la extinderea funcționalităților aplicației Mindful Wallet, oferind utilizatorilor instrumente și resurse adiționale pentru o gestionare financiară mai eficientă și mai informată

Bibliografie

- [1] Programare Frontend: Angular vs React (2024) Disponibil la: <https://www.newtech.ro/blog-react-vs-angular/> [Accesat 02 Iunie 2024].
- [2] Awati, R. & Hughes, A., n.d. Microsoft SQL Server. Disponibil la: <https://www.techtarget.com/searchdatamanagement/definition/SQL-Server> [Accesat 02 Iunie 2024].
- [3] Freeman, A. (2020) Getting Ready, în Pro Angular 9. 4th edn. London, UK: Apress, pp. 28–29.
- [4] Freeman, A. (2020) Putting Angular în Context, in Pro Angular 9. 4th edn. London, UK: Apress, pp. 69–72.
- [5] Lock, A. (2021) ASP.NET Core In Action. 2nd edn. Shelter Island, NY: Manning, pp. 46 .
- [6] Lock, A. (2021) ASP.NET Core In Action. 2nd edn. Shelter Island, NY: Manning, pp. 56-57.
- [7] Martin, R. C., (2012). The Clean Architecture. Disponibil la: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html> [Accesat 02 Iunie 2024].
- [8] Ray, M. & West, R., (2024). What is SQL Server?. Disponibil la: <https://learn.microsoft.com/en-us/sql/sql-server/what-is-sql-server?view=sql-server-ver16> [Accesat 02 june 2024].
- [9] Sahu, A., 2023. Implement Entity Framework A Code First Approach in .Net 8 API.Disponibil la: <https://medium.com/@codewithhanksahu/implement-entity-framework-a-code-first-approach-in-net-8-api-80b06d219373> [Accesat 04 June 2024].
- [10] Sharma, A. K., (2021). Evolution / History of ASP.NET. Disponibil la: <https://www.ittutorialswithexample.com/2021/03/evolution-history-of-aspnet.html> [Accesat 02 Iunie 2024].
- [11] Team, S., (2022). What is password hashing?. Disponibil la: <https://stytych.com/blog/what-is-password-hashing/> [Accesat 03 Iunie 2024].
- [12] Component-Based Architecture, Disponibil la: https://www.tutorialspoint.com/software_architecture_design/component_based_architecture.htm [Accesat 01 Iunie 2024].
- [13] Introduction to JSON Web Tokens. Disponibil la: <https://jwt.io/introduction> [Accesat 3 Iunie 2024].
- [14] Astrid Müller, Maithilee Joshi, Tobias A. Thomas, Excessive shopping on the internet: recent trends in compulsive buying-shopping disorder (2022).